# Software rétegek és a Board szintű absztrakció

Scherer Balázs



**Budapesti Műszaki és Gazdaságtudományi Egyetem (BME),**
**Mesterséges Intelligencia és Rendszertervezés Tanszék (MIT)**

© BME-MIT 2026

# Tartalom

- **Az eddig megismert szoftver megoldások korlátai**
- Board Support Package (nem feltétlen linuxi értelemben)
  - Miért szükséges a BSP
  - Jellemző absztrakciós megoldások
    - Define alapú absztrakció
    - Függvény API alapú absztrakció
    - Pointer struktúra alapú absztrakció
  - Ipari példák absztrakciós szintekre, software rétegekre
    - AUTOSAR
    - ST STM BSP
- Példa absztrakciós szintek meghúzására

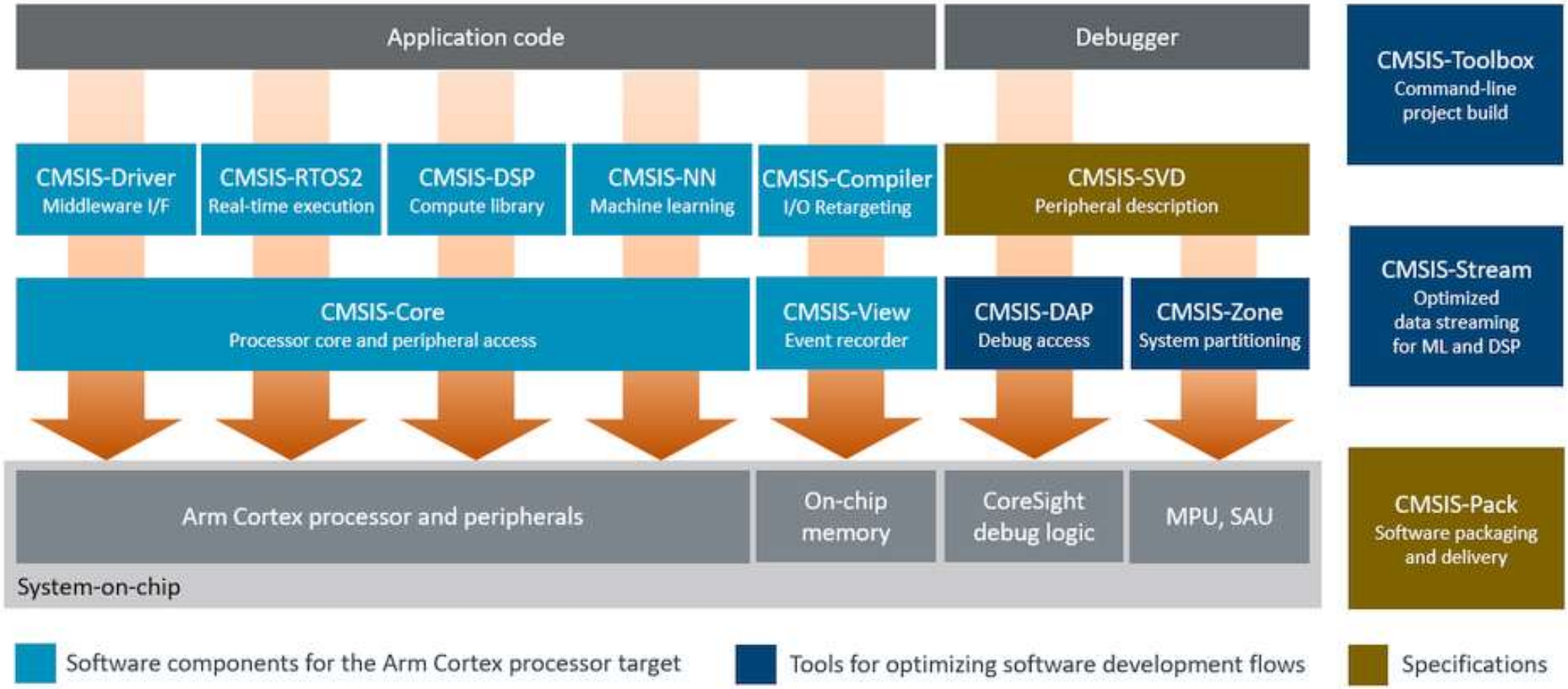

© BME-MIT 2026

# Mikrovezérlő szintű absztakció

- Eddig megismerkedtünk a mikrovezérlők gyártók által Firmware library-kkal, Jellemzően alacsony szintű periféria kezelési támogatást adnak
  - Általában hagyatkoznak a CMSIS alapokra
    - Elnevezési konvenció, Doxygen kommentezés …
  - Bár CMSIS javasol egy Driver library felületet, de ettől függetlenül tipikusan gyártó függők
- Megismerkedtünk a jellemző periféria kezelési módokkal:
  - Pollingos
  - Interrupt alapú
  - DMA alapú átvitel

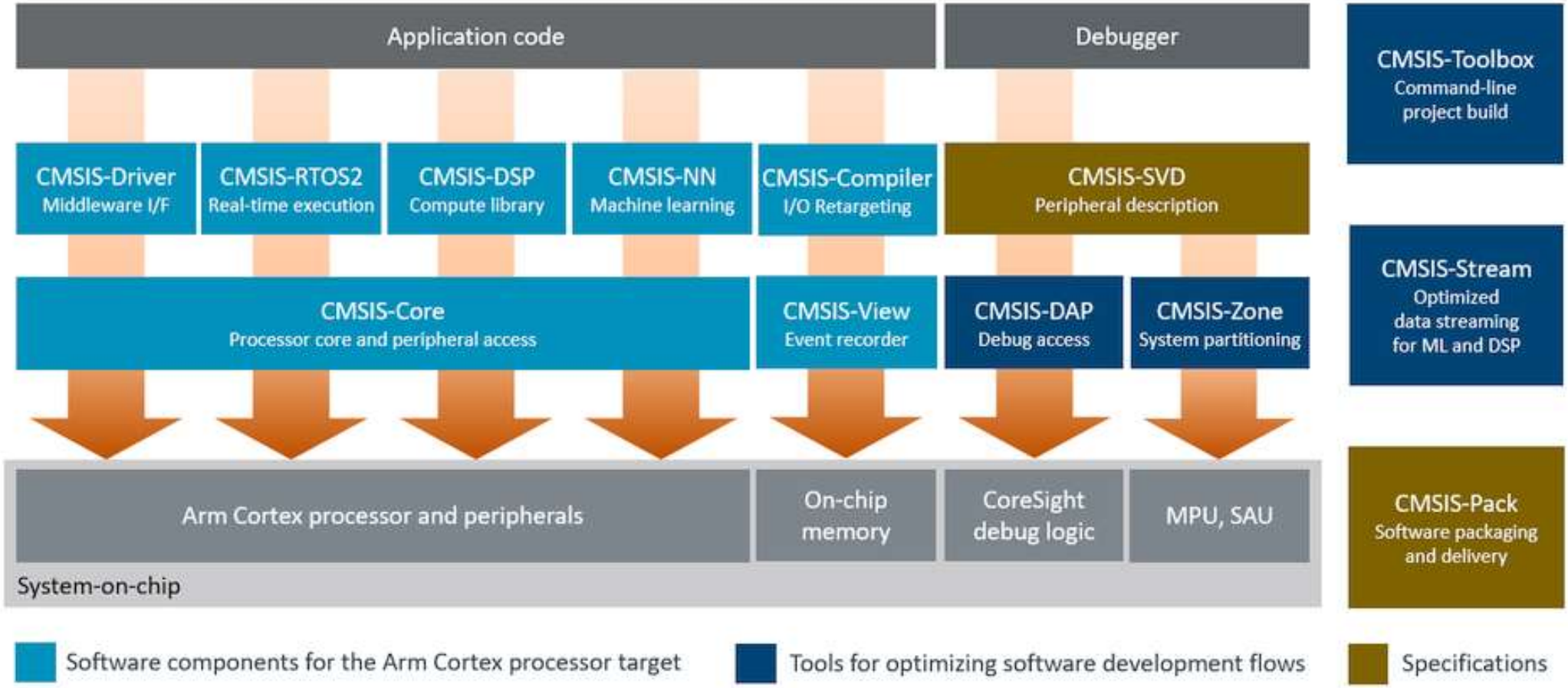

© BME-MIT 2026

# Mi van benne az eddig megismert CMSIS-ben?



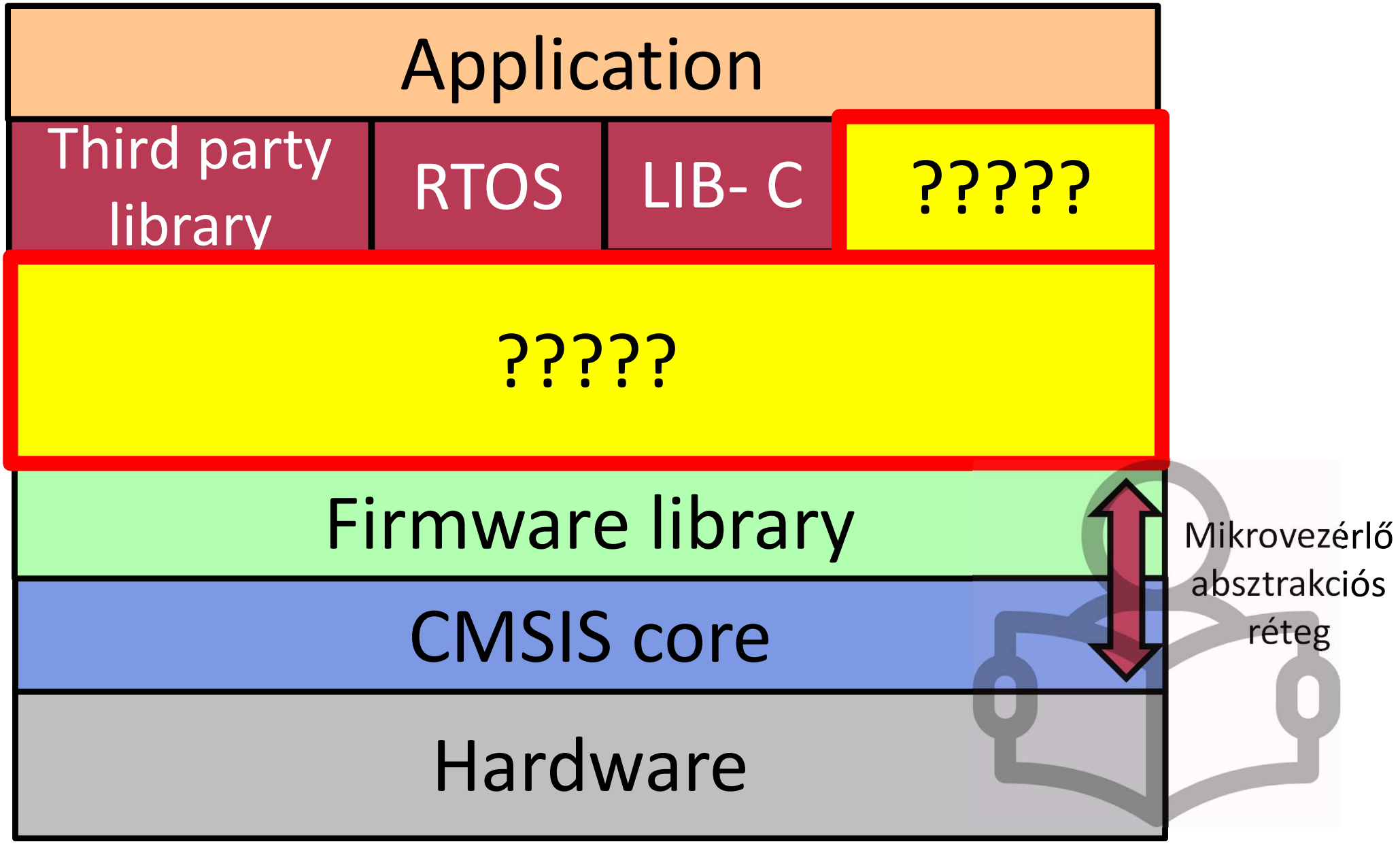

- Hiányzik ebből valami szerintünk?

# Mi van benne az eddig megismert CMSIS-ben?




- **Semmi olyan nincs benne, az adott NYÁK/board külső hardverének konkrét felépítését és funkcióját írná le.**

# Beágyazott rendszerek réteges szerkezete



# Tartalom

- Az eddig megismert szoftver megoldások korlátai
- **Board Support Package (nem feltétlen linuxi értelemben)**
  - **Miért szükséges a BSP**
  - Jellemző absztrakciós megoldások
    - Define alapú absztrakció
    - Függvény API alapú absztrakció
    - Pointer struktúra alapú absztrakció
  - Ipari példák absztrakciós szintekre, software rétegekre
    - AUTOSAR
    - ST STM BSP
- Példa absztrakciós szintek meghúzására


© BME-MIT 2026

# Miért szükséges a Board Support Package

- Az alap firmware library függvénykészletben semmi olyan támogatás **nincs, ami leírná, hogy a mikrovezérlőn kívül milyen dolgokat tartalmaz a NYÁK**
  - Nem felejtő memóriák
  - Szenzorok, beavatkozók
  - Kommunikációs interfészek
- A mikrovezérlők számtalan **perifériá**t tartalmaznak, de azok csak **kis részét használjuk** egy adott alkalmazásban
  - Melyik perifériák vannak és mire használva
  - Milyen konfigurációban és lábakkal vannak az adott perifériák használva



# Mit a BSP célja

- **Az összes mikrovezérlő periféria adott hardware-e való konfigurációját tartalmazza**
  - Periféria beállítás, órajel engedélyezés, lábkiosztás
- **A hardware-en található egyes fizikai komponensek számára könnyen kezelhető általános felületet adjon**
  - Nem felejtő memóriák
  - Szenzorok, beavatkozók
  - Kommunikációs interfészek
- A felsőbb rétegek számára **olyan funkciók, absztrakciók szolgáltatása, amelyek eltakarják a hardware-es megvalósítás részleteit**.



# Tartalom

- Az eddig megismert szoftver megoldások korlátai
- **Board Support Package (nem feltétlen linuxi értelemben)**
  - Miért szükséges a BSP
  - **Jellemző absztrakciós megoldások**
    - Define alapú absztrakció
    - Függvény API alapú absztrakció
    - Pointer struktúra alapú absztrakció
  - Ipari példák absztrakciós szintekre, software rétegekre
    - AUTOSAR
    - ST STM BSP
- Példa absztrakciós szintek meghúzására

# Miért kell absztrakciókat használni

- **Miért nem akarjuk az alkalmazásban látni, hogy?**

  ***GPIO_PinOutSet ( gpioPortF, 7)***

- **Mert nem tudjuk egy szint után, hogy ez** most egy LED bekapcsolása, vagy mondjuk az UART engedélyezése, vagy **mi, nem akar mindenki kapcsolási rajzot nézegetni** …
- **Az ilyen kód nem hordozható**: Változik a NYÁK elrendezés, új lábra kerül a funkció, minden helyen végig kellene vezetni a változást: követhetetlen és veszélyes
- **Tesztelhetőség**: az absztrakciós határokon egy réteg teszt csonkkal (stub/mock) helyettesíthető, így **a rétegek külön-külön tesztelhetők**



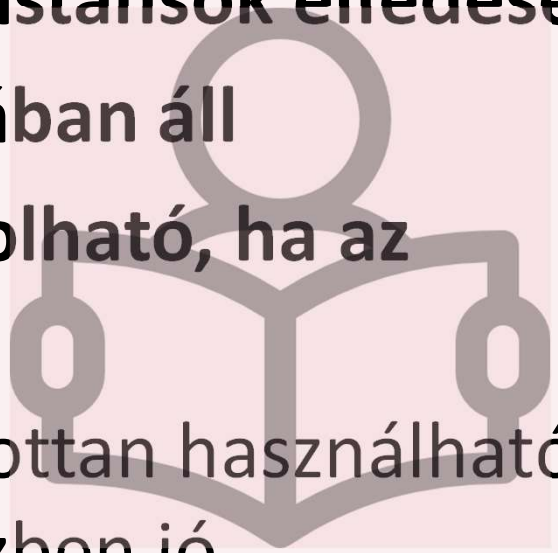
# Tartalom

- Az eddig megismert szoftver megoldások korlátai
- **Board Support Package (nem feltétlen linuxi értelemben)**
  - Miért szükséges a BSP
  - **Jellemző absztrakciós megoldások**
    - **Define alapú absztrakció**
    - Függvény API alapú absztrakció
    - Pointer struktúra alapú absztrakció
  - Ipari példák absztrakciós szintekre, software rétegekre
    - AUTOSAR
    - ST STM BSP
- Példa absztrakciós szintek meghúzására

# Define alapú absztrakció

**#define CIMKE FUNKCIO**

- **Egyszerű név helyettesítés:** alacsony szintű absztrakció
- A firmware libray-tól (mikrovezérlő absztrakciós réteg) való absztrakciót jellemzően nem oldja meg
- Tipikusan egyedi, érthetetlen **varázskonstansok elfedése**
- Ezen a szinten **jellemzően nem önmagában áll**
- **Board-változatok között könnyen portolható, ha az MCU/firmware API nem változik.**
- Bonyolultabb perifériákra csak korlátozottan használható: például UART0 absztrakciójára csak részben jó



# Példa Define alapú absztrakcióra: LED kezelés

**Header file, vagy lokális define-ok**

```
#define LED1_Port   gpioPortE
#define LED1_Pin    2
#define LED2_Port   gpioPortE
#define LED2_Pin    3
```

Absztrakt LED elnevezés

Hardware kapcsolat

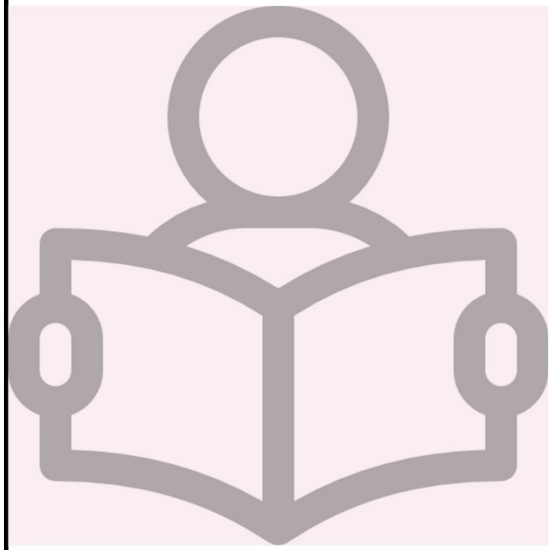
**C file**

„Hardware független” LED kezelés

```
CMU_ClockEnable (cmuClock_GPIO, true);

GPIO_PinModeSet (LED1_Port,LED1_Pin, gpioModePushPull,1);
GPIO_PinModeSet (LED2_Port,LED2_Pin, gpioModePushPull,1);

while(1)
{
    GPIO_PinOutToggle(LED1_Port, LED1_Pin);
    GPIO_PinOutToggle(LED2_Port, LED2_Pin);
}
```



# Példa Define alapú absztrakcióra: LED kezelés

**Header file, vagy lokális define-ok**

```
#define LED1_Port  gpioPortE
#define LED1_Pin   2
#define LED2_Port  gpioPortE
#define LED2_Pin   3
```

Továbbra is erősen függ a Firmware library függvényeitől. A USER magas szinten nem akar ilyet látni!

**C file**

```
CMU_ClockEnable (cmuClock_GPIO, true);

GPIO_PinModeSet (LED1_Port,LED1_Pin, gpioModePushPull,1);
GPIO_PinModeSet (LED2_Port,LED2_Pin, gpioModePushPull,1);

while(1)
{
    GPIO_PinOutToggle(LED1_Port, LED1_Pin);
    GPIO_PinOutToggle(LED2_Port, LED2_Pin);
}
```



# Példa Define alapú absztrakcióra: UART kezelés

Absztrakt UART elnevezés

**Header file, , vagy lokális define-ok**

Hardware kapcsolat

```
#define DIAG_UART__IOENABLE_PORT    gpioPortF
#define DIAG_UART__IOENABLE_PIN     7

#define DIAG_UART__UART             UART0
#define DIAG_UART__RX_PORT          gpioPortE
#define DIAG_UART__RX_PIN           1
#define DIAG_UART__TX_PORT          gpioPortE
#define DIAG_UART__TX_PIN           0
#define DIAG_UART__ROUTE_LOCATION   UART_ROUTE_LOCATION_LOC1
```

# Példa Define alapú absztrakcióra: UART kezelés

## Source file

```
CMU_ClockEnable (cmuClock_GPIO, true);
CMU_ClockEnable (cmuClock_UART0, true);

GPIO_PinModeSet (DIAG_UART__IOENABLE_PORT,DIAG_UART__IOENABLE_PIN, gpioModePushPull,1);

USART_InitAsync_TypeDef   uartInitStruct;
uartInitStruct.baudrate = 115200;
uartInitStruct.enable = usartEnable;
uartInitStruct.databits = usartDatabits8 ;
uartInitStruct.mvdis = false;
uartInitStruct.oversampling = usartOVS16;
uartInitStruct.parity = usartNoParity;
uartInitStruct.prsRxEnable = false;
uartInitStruct.stopbits = usartStopbits1;
uartInitStruct.refFreq = 0;

USART_InitAsync (DIAG_UART__UART, &uartInitStruct);

// Set TX and RX pins as push-pull output and input resp.
GPIO_PinModeSet(DIAG_UART__RX_PORT, DIAG_UART__RX_PIN, gpioModeInput, 1);
GPIO_PinModeSet(DIAG_UART__TX_PORT, DIAG_UART__TX_PIN, gpioModePushPull, 0);

// Enable both RX and TX for routing
DIAG_UART__UART->ROUTE |= DIAG_UART__ROUTE_LOCATION;
DIAG_UART__UART->ROUTE |= UART_ROUTE_TXPEN | UART_ROUTE_RXPEN;

USART_Tx (DIAG_UART__UART,'+');
```



Továbbra is erősen függ a Firmware library függvényeitől. De egy más UART-ra való áttérésnél elég a define-okat átírni

# Tartalom

- Az eddig megismert szoftver megoldások korlátai
- **Board Support Package (nem feltétlen linuxi értelemben)**
  - Miért szükséges a BSP
  - **Jellemző absztrakciós megoldások**
    - Define alapú absztrakció
    - **Függvény API alapú absztrakció**
    - Pointer struktúra alapú absztrakció
  - Ipari példák absztrakciós szintekre, software rétegekre
    - AUTOSAR
    - ST STM BSP
- Példa absztrakciós szintek meghúzására

# Függvény, függvény makró alapú absztrakció

- Adott feladathoz **saját API** (Application Programming Interface) **létrehozása, ami eltakarja a firmware library**-t.
- Célszerű **kombinálni a #define alapú megvalósítással**
- **Egy** adott **feladathoz tartozó API-t** célszerű **saját C és header file-ba** szervezni
- Az **API szerkezet**ét feladat függően hozzuk létre, jellemzően egyedi, **nincs rá szabály**ozás
- Nincs rá szabály, de **célszerű elnevezési konvenciót használni** hasonlóan a firmware library-hoz: PL__Init(), PL_SendData…
- Nincs rá szabály, de **célszerű komment alapú dokumentációt használni**. Pl.: Doxgen
- **Visszajelzés** vagy függvény **visszatérési értékkel, vagy CallBack** függvénnyel



# Példa Függvény alapú LED absztrakcióra

**Header file**

```
typedef enum
{
  LED1 = 0,
  LED2 = 1
} Led_TypeDef;

void LED__Init_LEDS(void);
void LED__Toggle(Led_TypeDef led);
void LED__ON(Led_TypeDef led);
void LED__OFF(Led_TypeDef led);
```

Fimware library és hardware független függvények



# Példa Függvény alapú LED absztrakcióra

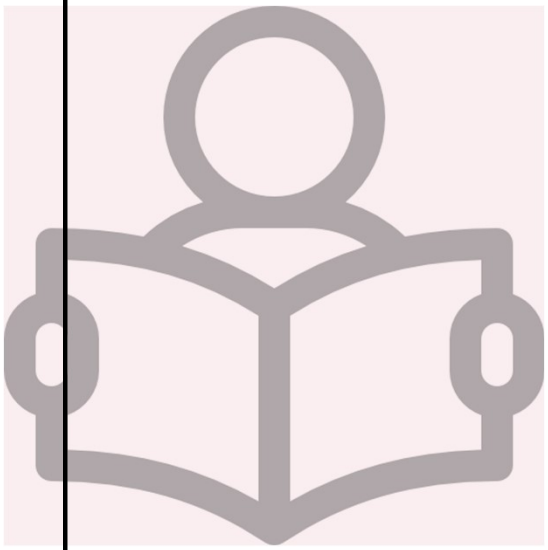
**C file**

```
#define LED1_Port gpioPortE
#define LED1_Pin  2
#define LED2_Port gpioPortE
#define LED2_Pin  3

void LED__Init_LEDS(void)
{
  CMU_ClockEnable (cmuClock_GPIO, true);
  GPIO_PinModeSet (LED1_Port,LED1_Pin, gpioModePushPull,0);
  GPIO_PinModeSet (LED2_Port,LED2_Pin, gpioModePushPull,0);
}
void LED__Toggle(Led_TypeDef led)
{
  switch(led)
  {
    case LED1:
        GPIO_PinOutToggle(LED1_Port, LED1_Pin);
        break;
    case LED2:
        GPIO_PinOutToggle(LED2_Port, LED2_Pin);
        break;
  }
}
```

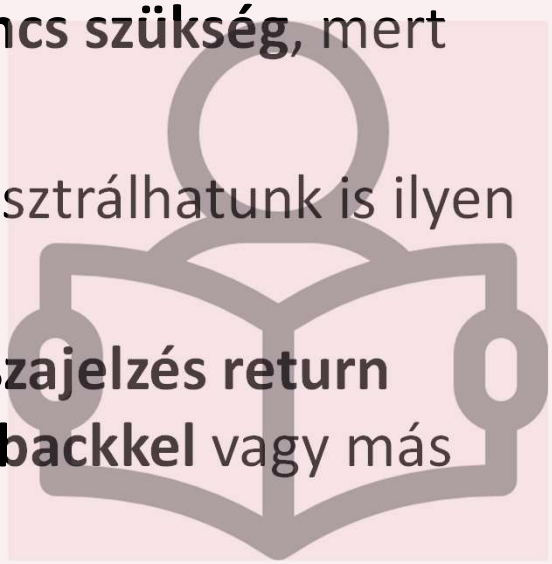
#define alapú portolás könnyítés

Hardware független egyszerű kezelőfüggvények

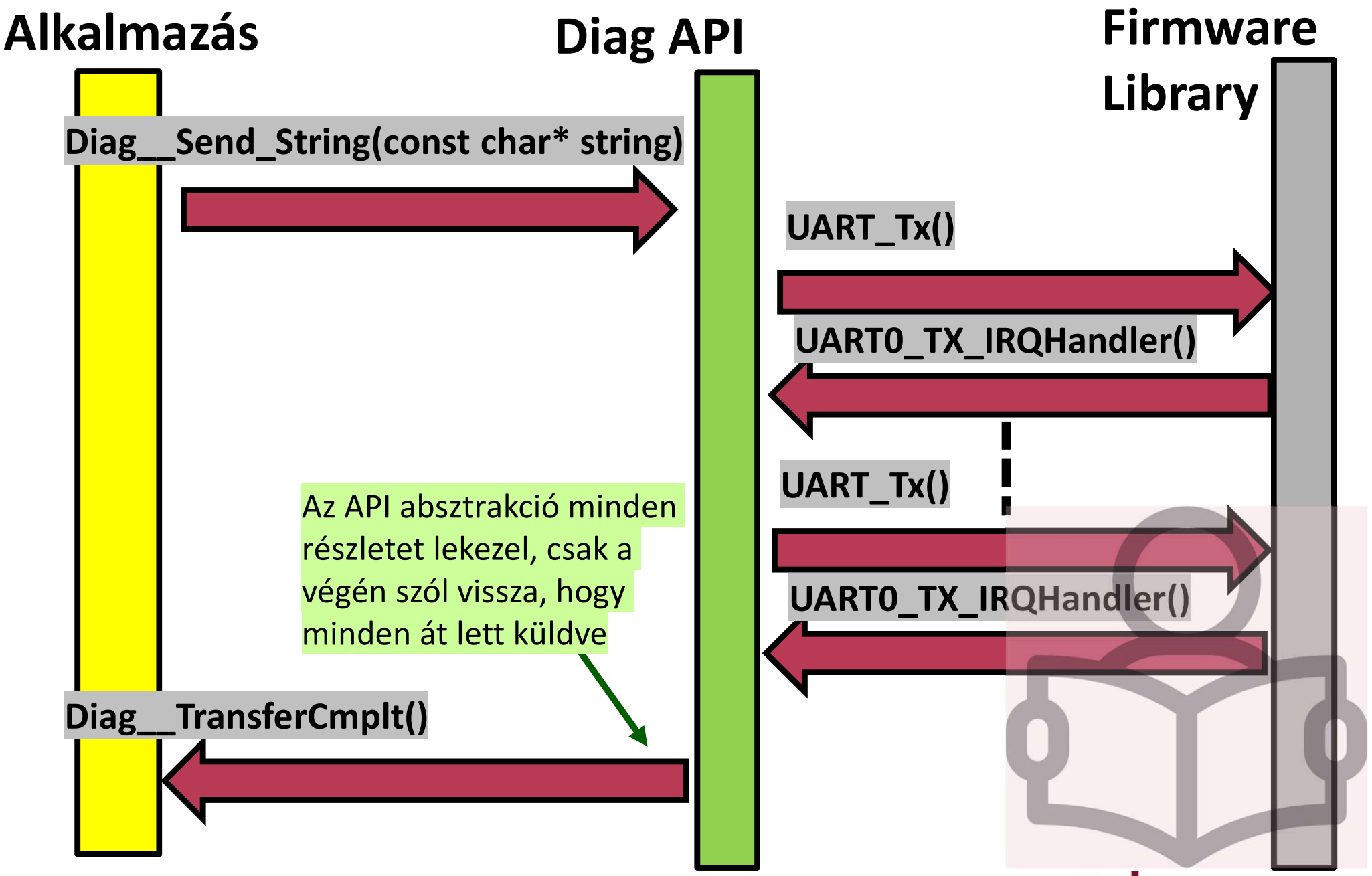


# CallBack függvények

- A legtöbb magas szintű **API  CallBack függvényen keresztülkeresztül jelez vissza**
- **CallBack függvény**eket akkor használunk, ha a rendszerben valamilyen **eseményről akarjuk a felsőbb réteget**: az alkalmazást / usert **értesíteni**
- A **CallBack** jellemzően **hardware független:** Ne kelljen magasabb szintről direktben IT-t kezelni
- A **CallBack sokszor Interrupt kontextusban van**, de nem feltétlenül direktben IT hívás: **Az Interuptokra tartozó szabályokat** itt is **be kell tartani**, de jellemzően **direkt hardware kezelésre nincs szükség**, mert hardware független (nyugtázás)
- A CallBack függvény lehet előre fixált nevű, vagy regisztrálhatunk is ilyen függvényt a megvalósítás függvényében
- Általános szabály: **Szinkron művelet: eredmény/visszajelzés return value-val**; **aszinkron művelet: későbbi esemény callbackkel** vagy más eseményjelzéssel.



# Péda CallBack függvény használatára

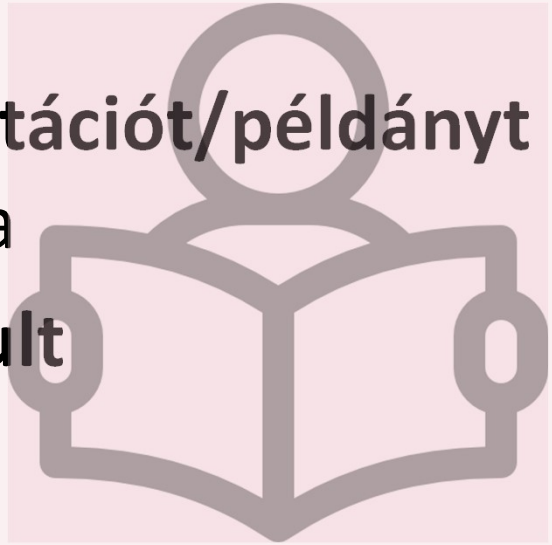


# Tartalom

- Az eddig megismert szoftver megoldások korlátai
- **Board Support Package (nem feltétlen linuxi értelemben)**
  - Miért szükséges a BSP
  - **Jellemző absztrakciós megoldások**
    - Define alapú absztrakció
    - Függvény API alapú absztrakció
    - **Pointer struktúra alapú absztrakció**
  - Ipari példák absztrakciós szintekre, software rétegekre
    - AUTOSAR
    - ST STM BSP
- Példa absztrakciós szintek meghúzására

# Pointer struktúra alapú absztrakció

- Az adott **funkciót** a **konfigurációs adataival** és vezérlő **függvényeivel** együtt **egységbe zárjuk**
- Nagyon hasonlít az **objektum orientált szemléletre**
- Sokszor létrehoznak egy „**absztakt class-t**” és azt „**példányosítjuk**”
- **Legjobban hardware függetlenné** tehető és **legkönnyebben portolható** megoldás
- **Azonos interfész mögött több implementációt/példányt** is kezelhetsz, akár futásidőben kiválasztva
- **Sokszor nehezen érthető** és elég **bonyolult**
- Sok gyári BSP használja



# Példa LED-ek függvény pointer alapú kezelésére Silabs: blink_baremetal

- Silabs emlib driver függvények magas szinten simple LED-et mint absztrakt „class” definiálják

**sl_led.h**

„Absztrakt class definició”

```
typedef uint32_t sl_status_t;

#define SL_LED_CURRENT_STATE_OFF  0U              ///< LED state off
#define SL_LED_CURRENT_STATE_ON   1U              ///< LED state on
typedef uint8_t sl_led_state_t;                 ///< LED state
/// A LED instance
typedef struct {
  void            *context;                     ///< The context for this LED instance
  sl_status_t     (*init)(void *context);       ///< Member function to initialize LED instance
  void            (*turn_on)(void *context);    ///< Member function to turn on LED
  void            (*turn_off)(void *context);   ///< Member function to turn off LED
  void            (*toggle)(void *context);     ///< Member function to toggle LED
  sl_led_state_t (*get_state)(void *context);   ///< Member function to retrieve LED state
} sl_led_t;
```

# Példa LED-ek függvény pointer alapú kezelésére Silabs: blink_baremetal

- Simple LED példányosítási példa (picit egyszerűsítve)

**sl_led.h**

```
sl_simple_led_context_t simple_led0_context = {
  .port = gpioPortE,
  .pin =  2,
  .polarity = SL_SIMPLE_LED_POLARITY_ACTIVE_HIGH,
};

const sl_led_t sl_led_led0 = {
  .context = &simple_led0_context,
  .init = sl_simple_led_init,
  .turn_on = sl_simple_led_turn_on,
  .turn_off = sl_simple_led_turn_off,
  .toggle = sl_simple_led_toggle,
  .get_state = sl_simple_led_get_state,
};
```

Milyen porton, milyen lábon van a LED, 0-ra, vagy 1-re világí. **Ezek lesznek a „tagváltozók”** (valóságban a silabsnál itt még define-okat használnak hasonlóan az előzőekben bemutatottakhoz)

Tagváltozó

Metódusok megadása



# Példa LED-ek függvény pointer alapú kezelésére Silabs: blink_baremetal

- Példa a megvalósított metódusokra (picit egyszerűsítve)

**sl_simple_led.c**

```
sl_status_t sl_simple_led_init(void *context)
{
  sl_simple_led_context_t *led = context;
  CMU_ClockEnable(cmuClock_GPIO, true);
  GPIO_PinModeSet(led->port, led->pin, gpioModePushPull, !led->polarity);

  return SL_STATUS_OK;
}

void sl_simple_led_turn_on(void *context)
{
  sl_simple_led_context_t *led = context;
  if (led->polarity == SL_SIMPLE_LED_POLARITY_ACTIVE_LOW) {
    GPIO_PinOutClear(led->port, led->pin);
  } else {
    GPIO_PinOutSet(led->port, led->pin);
  }
}
```



# Példa LED-ek függvény pointer alapú kezelésére Silabs: blink_baremetal

- Felhasználása a kódban (egyszerűsítve, mert még egy külön API-t létrehoz a valóságban erre)

**app.c**

```
sl_led_led0.init(sl_led_led0.context);
sl_led_led0.turn_on(sl_led_led0.context);
```



# Pointer struktúra alapú absztrakció jellemzői

- Egy egyszerű LED-re valószínűleg nem éri meg …
- Viszont bonyolultabb dolgokra igen:
  - Szeretnénk hőmérséklet mérést használni egy alkalmazásban több helyen is
  - Az alkalmazás szempontjából nem érdekel minket, hogy ez melyik esetben hogy lesz megvalósítva
  - Létrehozunk egy absztrakt classt: Init, GetTemp, egyéb függvényekkel
  - Minden esetben úgy implementáljuk az adott hőmérőt ahogy a szituáció megkívánja
    - Lehet a hőmérő LM75 I2C interfésszel
    - Lehet a hőmérő PT100 ADC-n keresztül
    - Lehet a hőmérő belső hőmérő a uC-n belül.



# Tartalom

- Az eddig megismert szoftver megoldások korlátai
- Board Support Package (nem feltétlen linuxi értelemben)
  - Miért szükséges a BSP
  - Jellemző absztrakciós megoldások
    - Define alapú absztrakció
    - Függvény API alapú absztrakció
    - Pointer struktúra alapú absztrakció
  - **Ipari példák absztrakciós szintekre, software rétegekre**
    - **AUTOSAR**
    - ST STM BSP
- Példa absztrakciós szintek meghúzására

# AUTOSAR

- AUTOSAR: **AUT**omotive **O**pen **S**ystem **AR**chitecture
  www.autosar.org

- Iparági fejlesztési partnerség, amit autóipari fejlesztők és tool készítők hoztak létre.

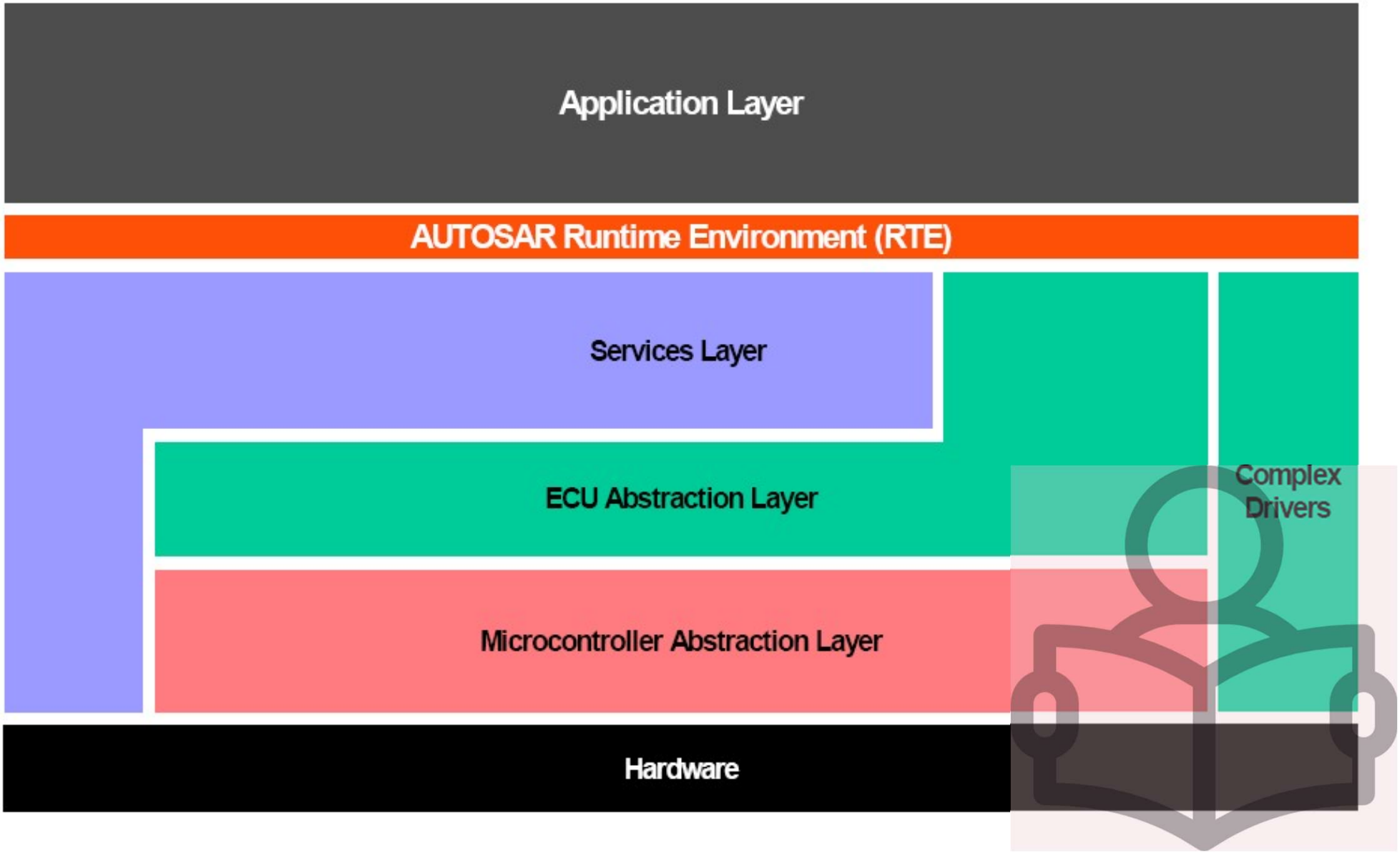
- Első release: 2005
- Célok:
  - Olyan utak keresése, hogy az elektronikus eszközök megbízhatóbbak és biztonságosabbak legyenek.
  - Egységes szabványkialakítást biztosító testület: „Cooperate on standards, compete on implementation”.
  - Biztosítsa a gyorsan növekvő technológiai bonyolultság csökkentését a tervezésnél, miközben lehetővé teszi az új technológiák integrálását.
  - Elősegíteni az autóba beépítendő szoftverek és hardverek hordozhatóságát, életciklus alatti karbantarthatóságát, illetve szoftver update/upgrade-et.

# AUTOSAR

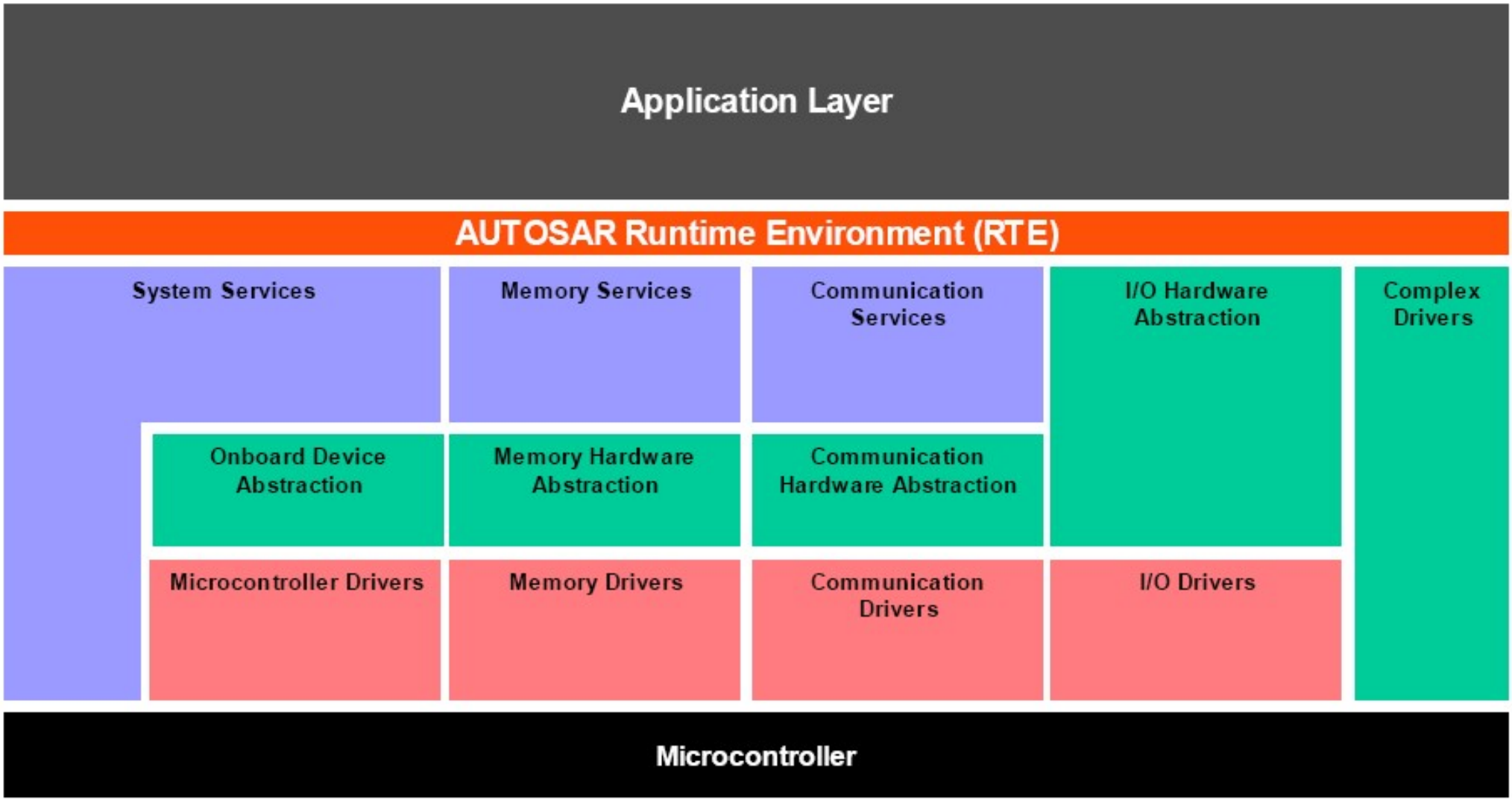
- **Függvény API alapú absztrakció, ahol legtöbb esetben a függvények neve erősen specifikálva van.** (Az eddig látottakhoz hasonlóan erősen szabványosított interfészekre épít, de az AUTOSAR ennél jóval több: architektúra, konfigurációs modell és fejlesztési módszertan is.)
- **Csak a filozófiára vagyunk kíváncsiak, nem a részletekre**
  - Erősen iparág specifikus
  - Nagyon komplex és bonyolult ( >10000 oldal doksi)
  - Nagyon épít az automatizálásokra és a toolokra
  - Akit érdekel van egy választható tárgy erről
  - Aki érdekel a legújabb verzió réteges szerkezetének áttekintése:

https://www.autosar.org/fileadmin/standards/R23-11/CP/AUTOSAR_CP_EXP_LayeredSoftwareArchitecture.pdf

# Réteges szemléletű szoftver architektúra



# Szoftver rétegek részletesebben (szándékosan picit régebbi, hogy egyszerűbb legyen)



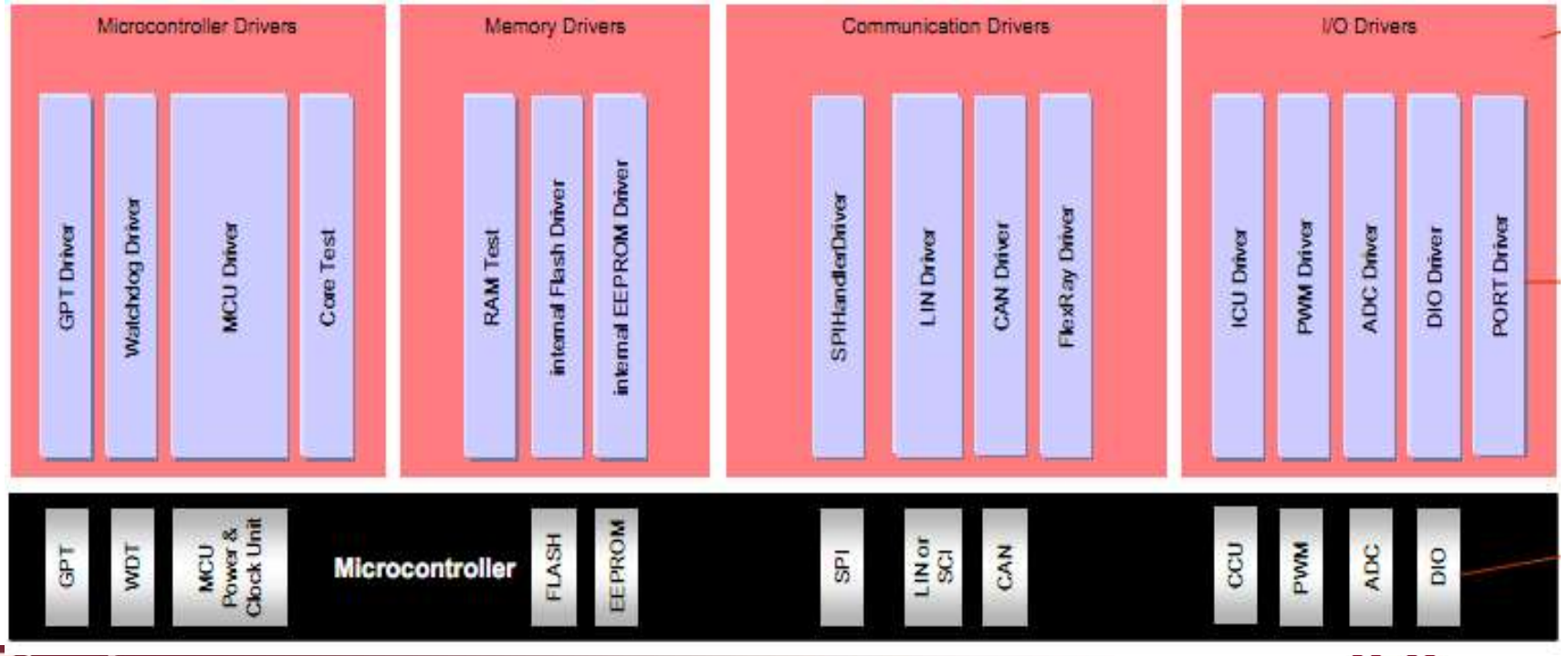
# Microcontroller Abstraction Layer (MCAL)

- **Szerepében nagyon hasonlít az általunk eddig látott firmare library-kal**
- Nagy különbség, hogy gyártó független
  - Emiatt jobban hasonlít a CMSIS-Drivers-re
- **Célja, hogy a mikrovezélő könnyen leváltható legyen**
  - Az elmúlt pár évtizedben számtalan chip válsága volt (legalább 3), erősen igazolta a létjogosultságát



# Microcontroller Abstraction Layer (MCAL)

- **Communication Drivers:**
  - Mikrovezérlő kommunikációs driverek (CAN, LIN etc.)
  - Egyébb NYÁK-on belüli kommunikációs driverek: SPI, I2C stb.
- **I/O Driverek**
  - Digitális és analóg I/O meghajtók (ADC, PWM, DAC)
- **Memória driverek**
  - Belső EEPROM/Flash kezelés alacsonyszintű függvényei
- **Microcontrel Drivers**
  - Watchdog, Timer kezelés
  - Beépített önteszt driverek stb.



# ECU absztrakciós réteg

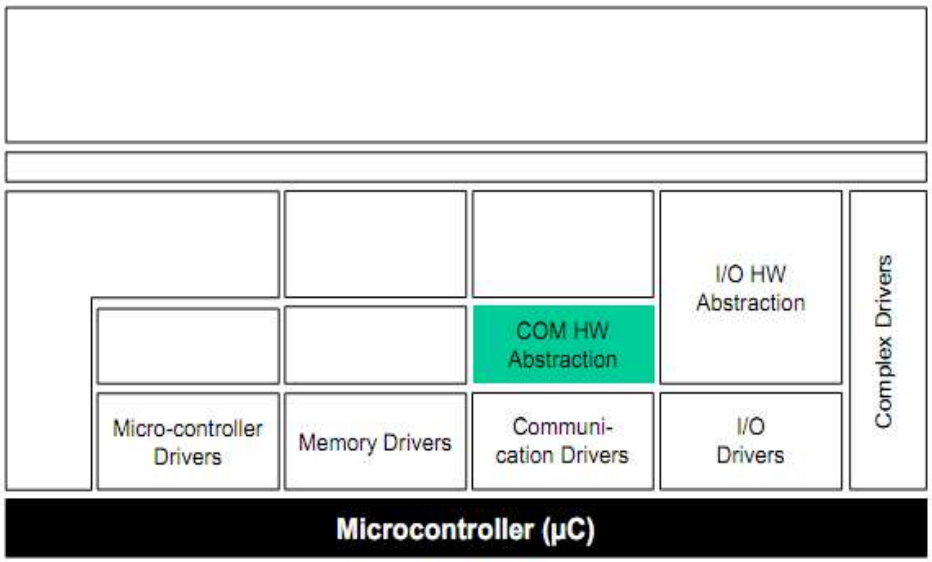
- Szoftver interfész az ECU elektronikus részéhez
- Elfedi a mikrokontrollert és a hardware specifikus funkcionalítást
- Célja, hogy az adott funkcióhoz az elektronikus megvalósításától, és a mikrovezérlőhöz való csatlakozásának a módjától függetlenül standard interfészt nyújtson

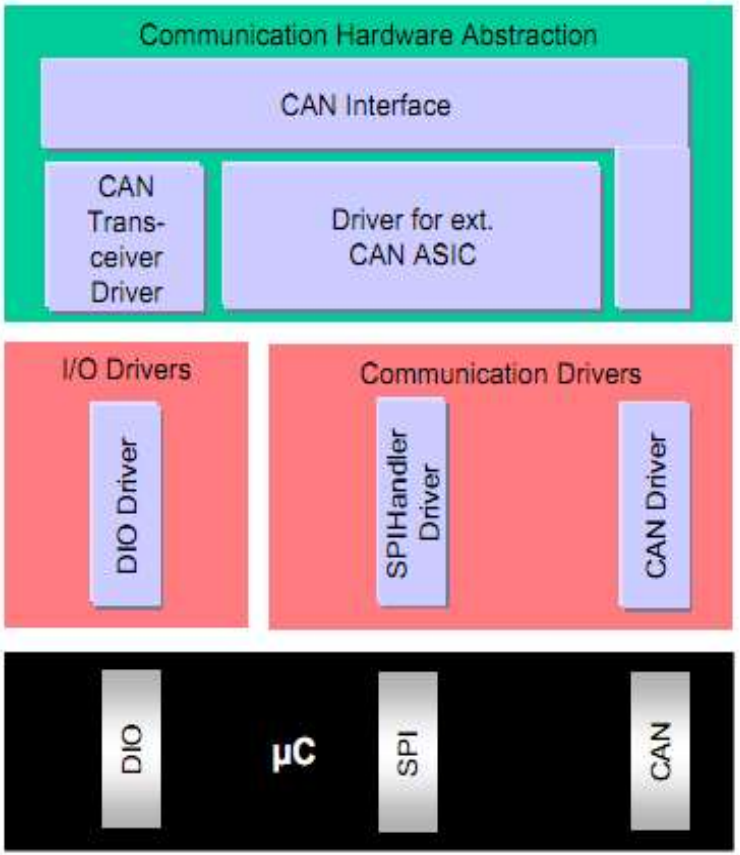


- **Azért lehet ezt elég hatékonyan megtenni, mert sok esetben elég konform, hogy milyen dolgokra van itt szükség**

# Communication Hardware Abstraction

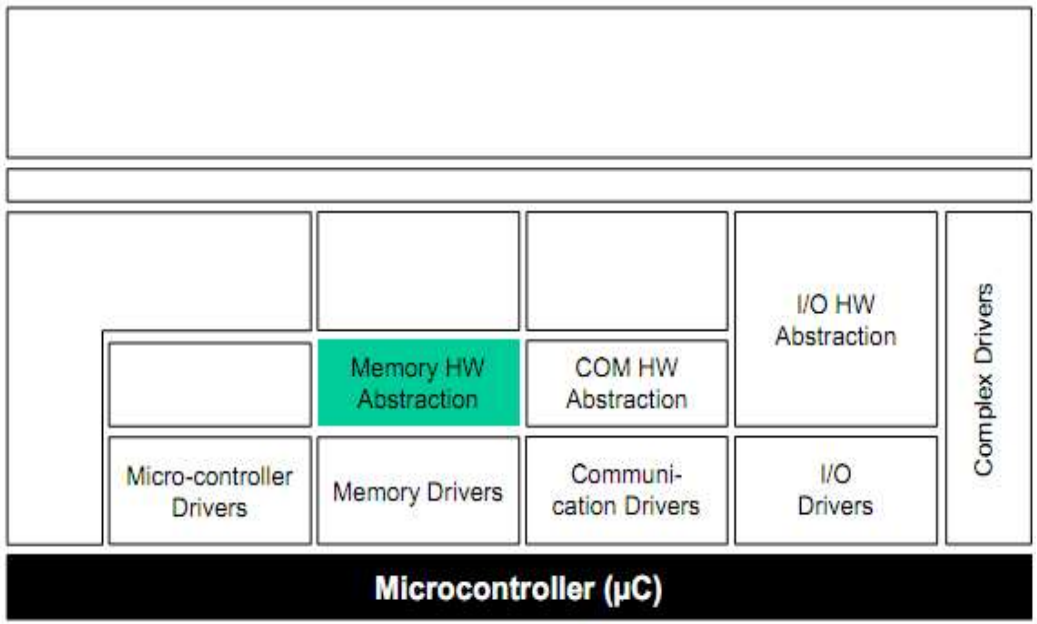
- **Communication Hardware Abstraction:**
  - Elfedi a kommunikációs kontrollerek helyét és ECU-n belüli elrendezését.
  - Minden kommunikációs rendszernek specifikus hardware absztrakciója van (CAN, LIN, FlexRay stb.)
  - Pl.: A mikrovezérlőnek van 2 belső és 4 külső CAN kontrollere, ezek a felsőbb szintek felé már egységesen látszódnak.
  - Mikrovezérlő független, de ECU hardware függő implementáció.
  - A felsőbb rétegeknek mutatott interfész ECU, és mikrovezérlő független, de kommunikációs busz függő.

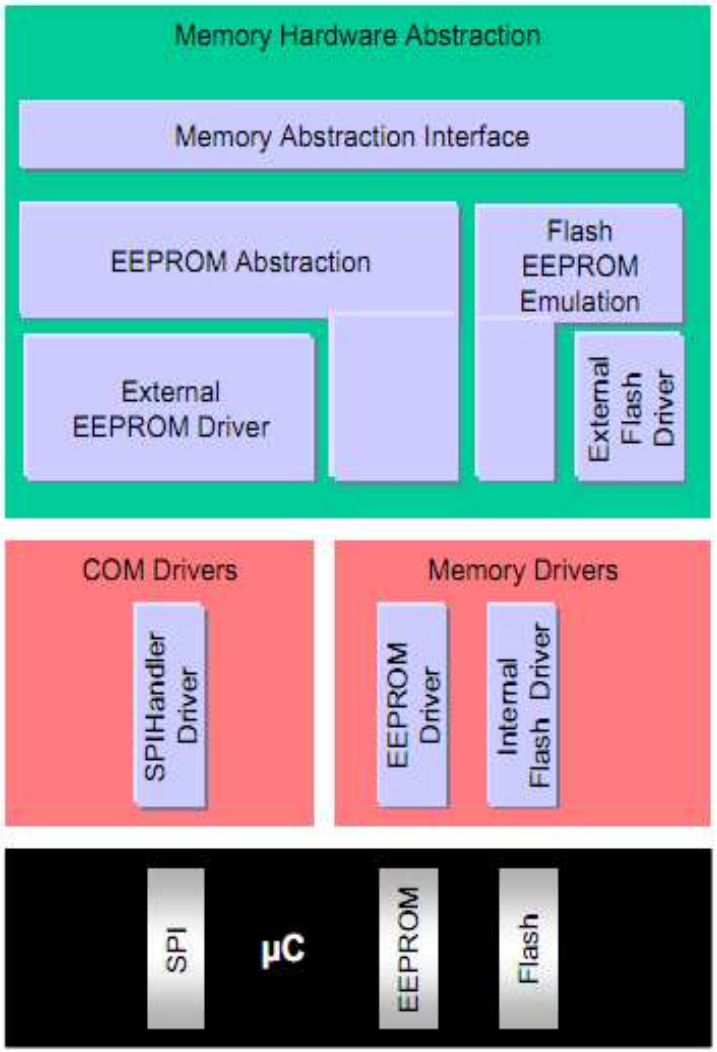


# Memory Hardware Abstraction

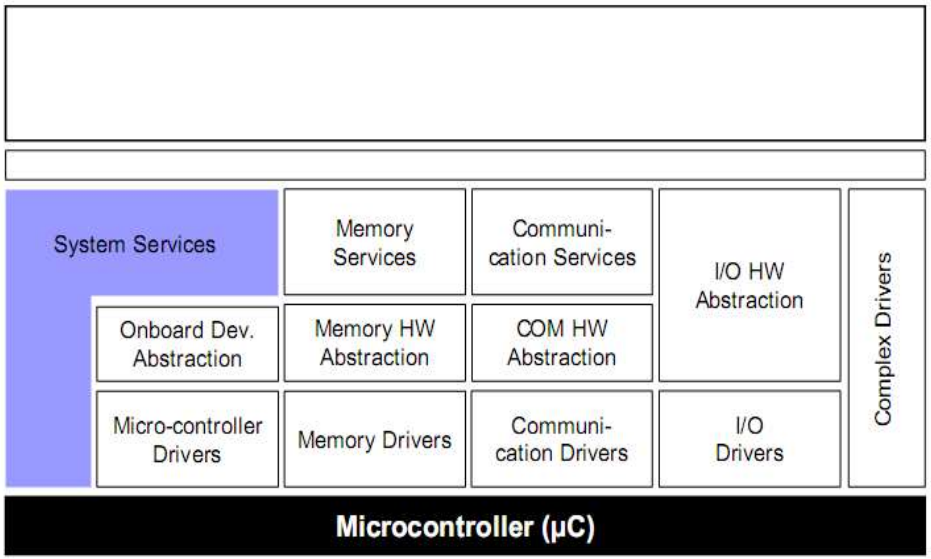
- **Memory Hardware Abstraction :**
  - Elfedi a periféria memória egységek helyét és ECU hardware layout-ját.
  - Magára a memória típusra és ad egy absztrakciót.
  - Az implementáció mikrovezérlő független, de külső memória függő.
  - A magasabb rétegeknek nyújtott interfész mikrovezérlő és ECU független

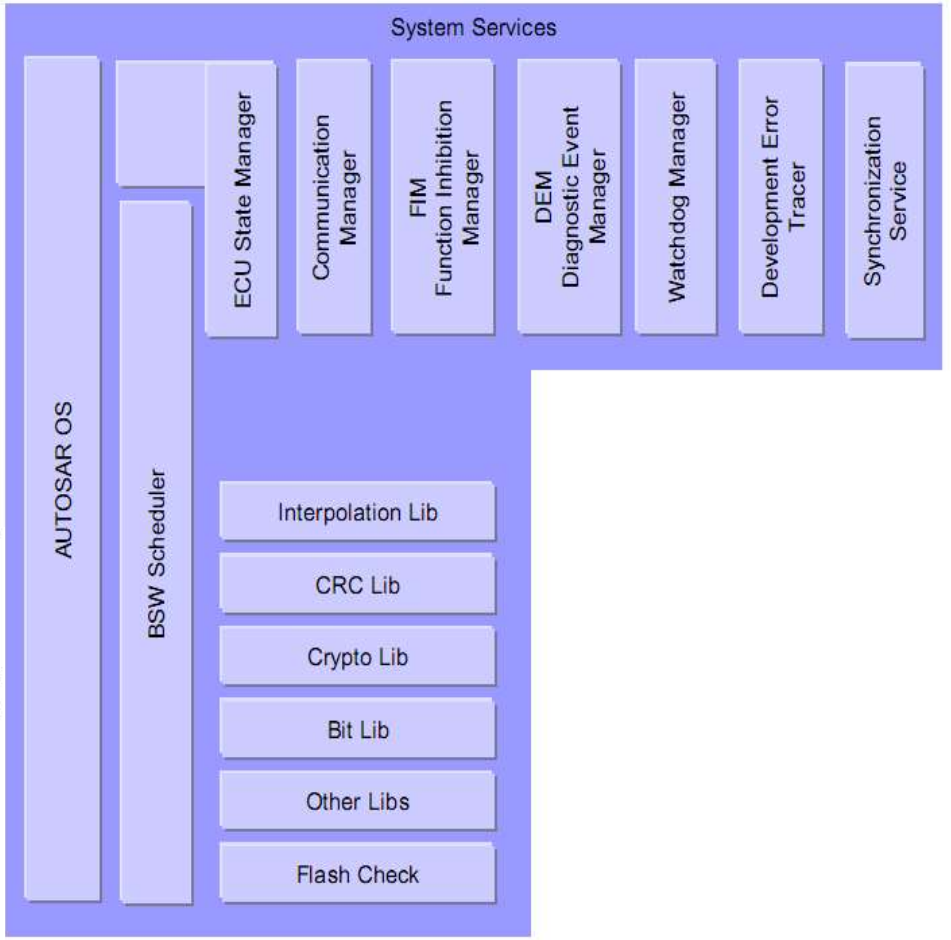


# System Services

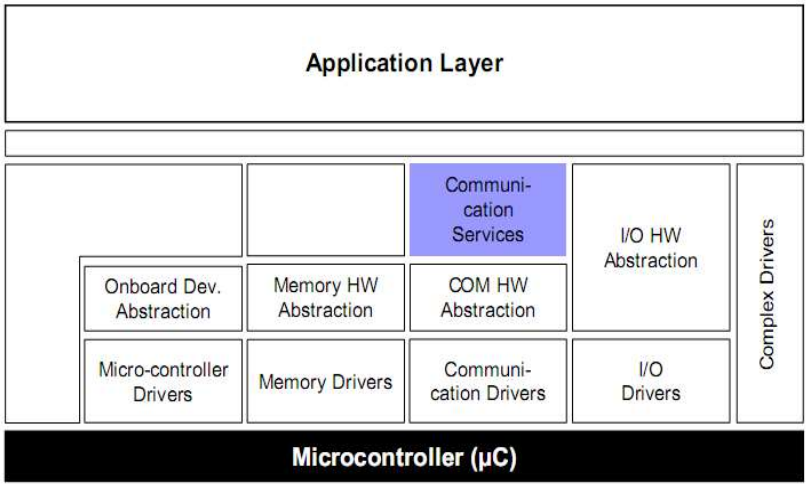
- **System services**
  - Olyan rendszer funkciókat tartalmaz, amelyeket minden modul használhat attól függetlenül, hogy melyik absztrakciós rétegben van.
  - RTOS szolgáltalások
  - Library szolgáltatások (CRC stb.)
  - Az implementáció mikrovezérlő és ECU valamint alkalmazás specifikus is lehet
  - Az RTE felé nyújtott interfész mikrovezérlő és ECU független

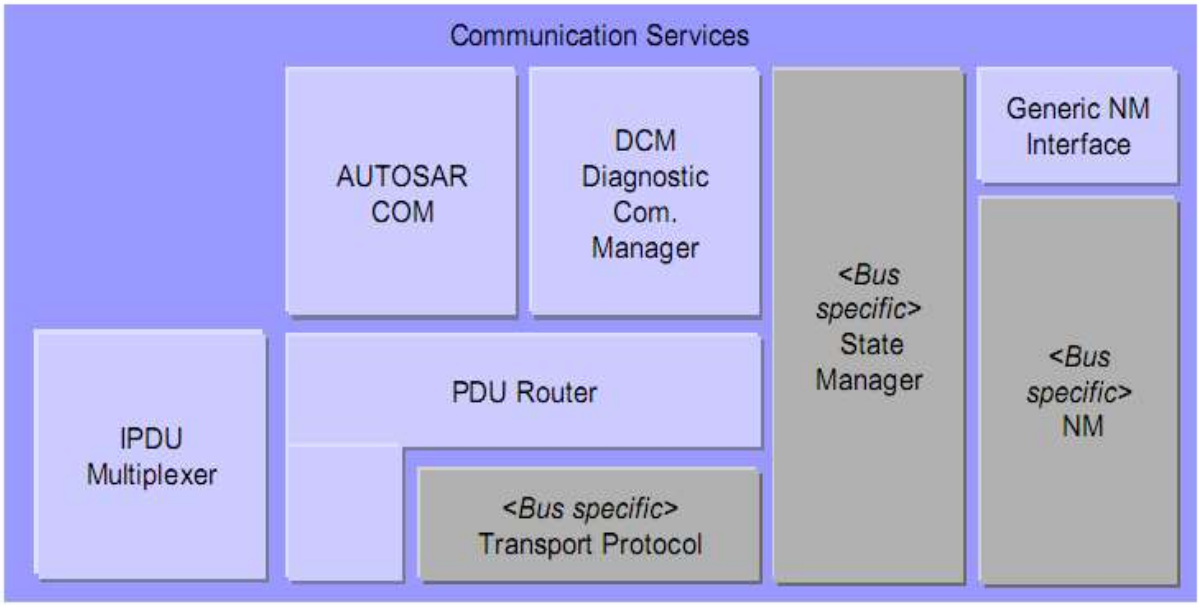


# Communication Services

- **Communication Services**
  - Egységes interfészt nyújt a kommunikációhoz
  - Egységes hálózatmenedzsment szolgáltatás
  - Egységes diagnosztikai kommunikáció
  - A kommunikációs protokoll tulajdonságainak elfedése az alkalmazás elöl
  - Mikrovezérlő és ECU hardware független az implementáció, ami azért függ a busz típusától.
  - A szolgáltatott interfész ECU, mikrokontroller és busz független



# Példák: Memória kezelés

This example shows how the NVRAM Manager and the Watchdog Manager interact with drivers on an assumed hardware configuration:

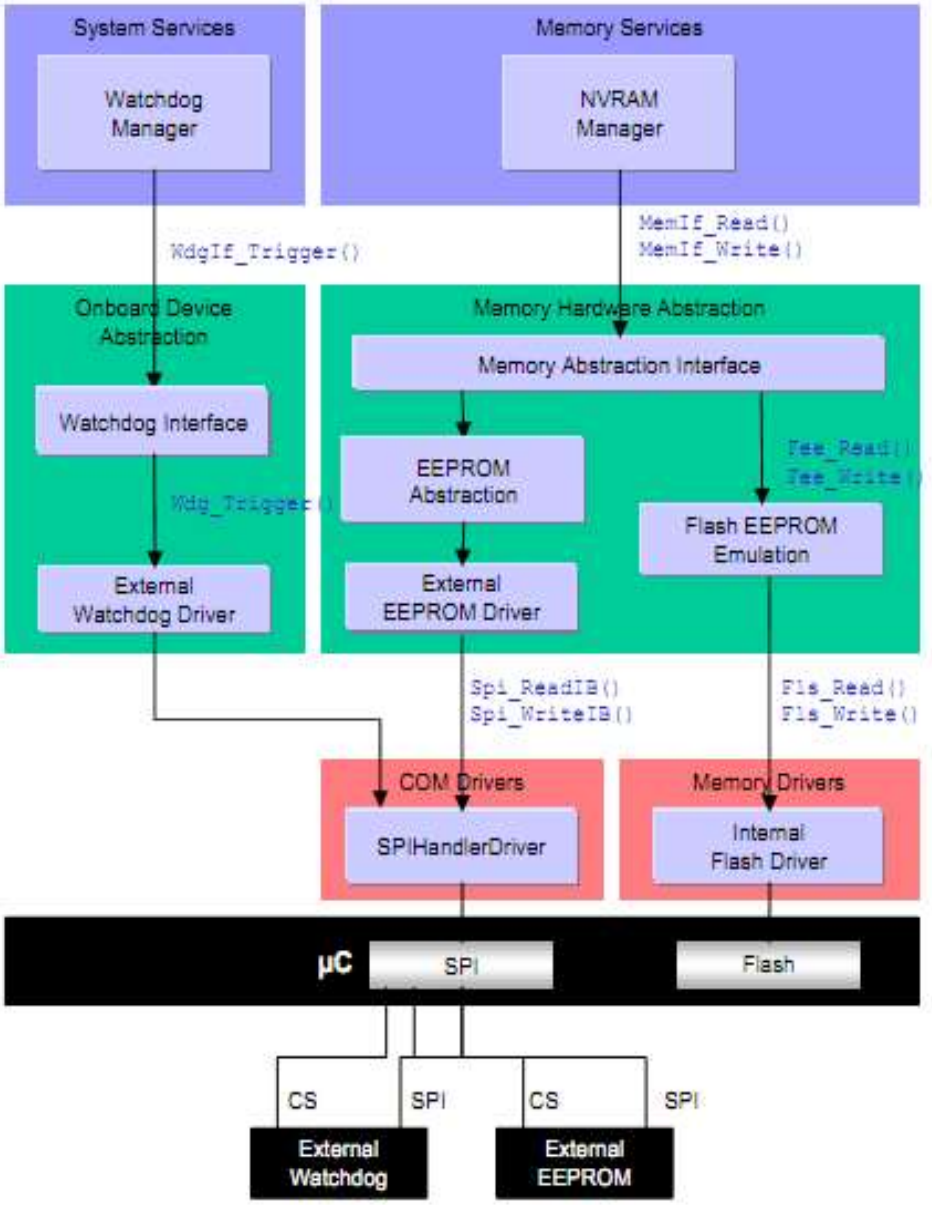
The ECU hardware includes an external EEPROM and an external watchdog connected to the microcontroller via the same SPI.

The SPIHandlerDriver controls the concurrent access to the SPI hardware and has to give the watchdog access a higher priority than the EEPROM access.

The microcontroller includes also an internal flash which is used in parallel to the external EEPROM. The EEPROM Abstraction and the Flash EEPROM Emulation have an API that is semantically identical.

The Memory Abstraction Interface can be realized in the following ways:

- routing during runtime based on device index (int/ext)
- routing during runtime based on the block index (e.g. > 0x01FF = external EEPROM)
- routing during configuration time via ROM tables with function pointers inside the NVRAM Manager (in this case the Memory Abstraction Interface only exists „virtually“)

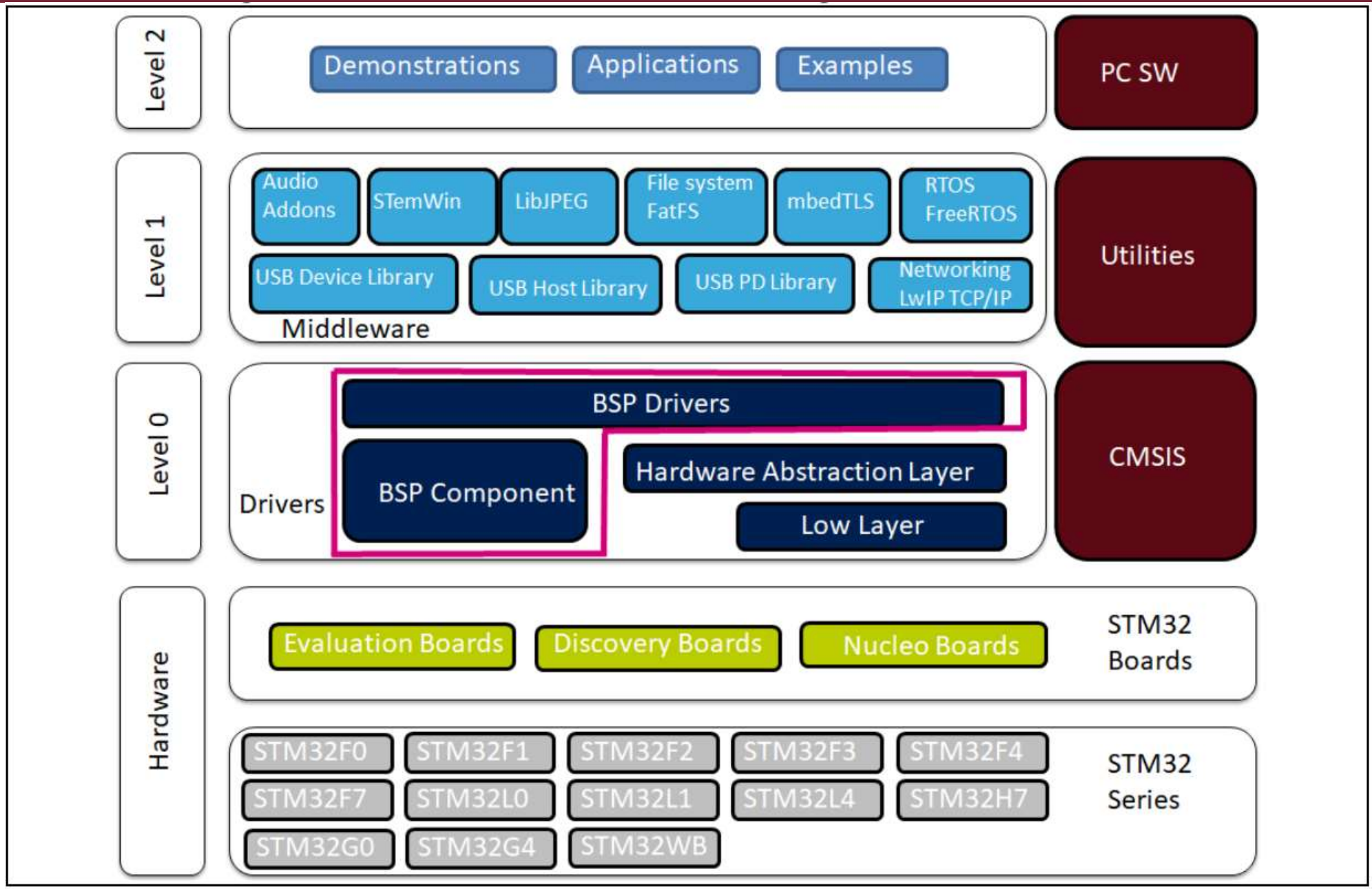


# Tartalom

- Az eddig megismert szoftver megoldások korlátai
- Board Support Package (nem feltétlen linuxi értelemben)
  - Miért szükséges a BSP
  - Jellemző absztrakciós megoldások
    - Define alapú absztrakció
    - Függvény API alapú absztrakció
    - Pointer struktúra alapú absztrakció
  - **Ipari példák absztrakciós szintekre, software rétegekre**
    - AUTOSAR
    - **ST STM BSP**
- Példa absztrakciós szintek meghúzására


© BME-MIT 2026

# BSP felépítés példa ST Microelectronics



# BSP felépítés példa ST Microelectronics

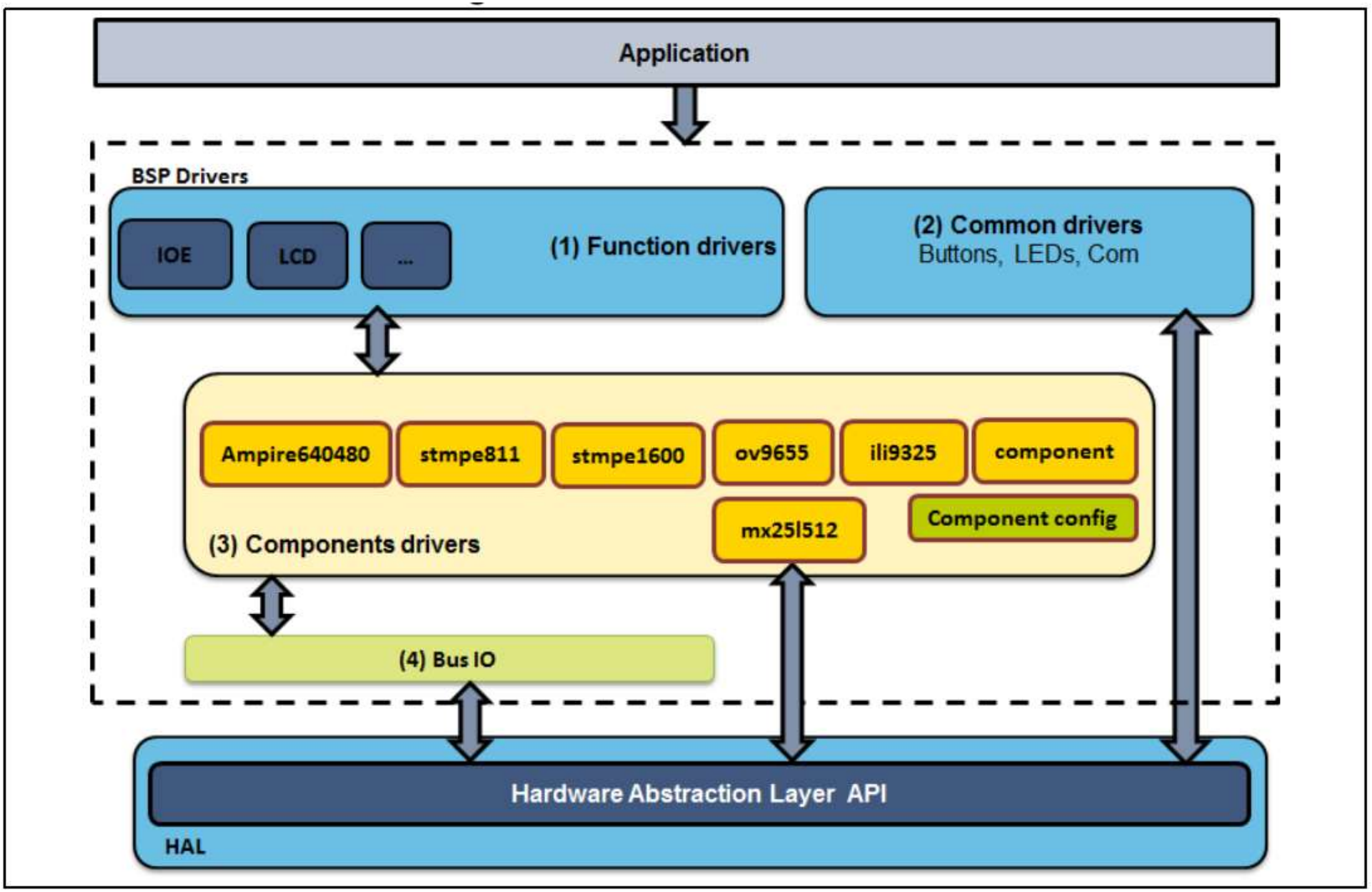
- Látható, hogy a BSP a HAL-ra hardware abstraction layer-re épül, ami az AUTOSAR microcontroller abstraction layernek, illetve a Silabs emlib-jéntek, emdrivers-énekfelel meg.

- A third party ún. middleware közvetlenül alkalmazás támogató library-k sokszor használják a BSP funkcionalitást, és nem direktben az alacsony szintű függvényeket használja


© BME-MIT 2026

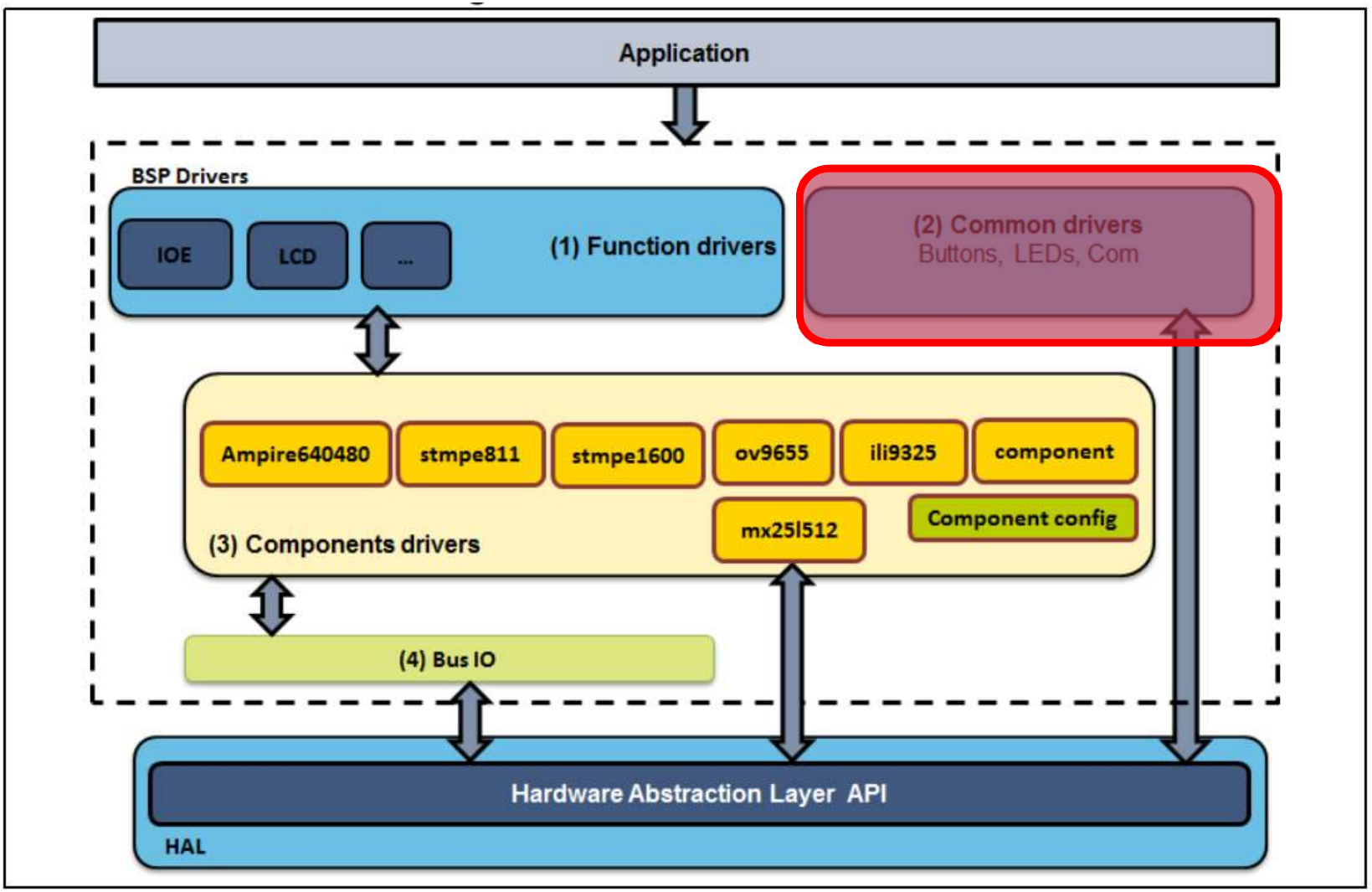
# BSP felépítés részletes példa ST Micro

- Látható, hogy itt is több szintre tagolódik maga a BSP funkcionalítás



# BSP felépítés részletes példa ST Micro

- Látható, hogy itt is több szintre tagolódik maga a BSP funkcionalítás



# BSP Common drivers ST Micro

- Common drivers
  - LED
  - Button
  - Comm
  - Potmeter
  - Joystic

- Célja, hogy a fejlesztő kártyák egyszerű kezelőfelületéhez hozzáférést nyújtson pl.:

# BSP Common drivers ST Micro

| Service | Description |
|---|---|
| int32_t BSP_GetVersion (void) | Return BSP version |
| int32_t BSP_LED_Init (Led_TypeDef Led) | Initialize a led |
| int32_t BSP_LED_DeInit (Led_TypeDef Led) | De-initialize a led |
| int32_t BSP_LED_On (Led_TypeDef Led) | Turn On a led |
| int32_t BSP_LED_Off (Led_TypeDef Led) | Turn Off a led |
| int32_t BSP_LED_Toggle (Led_TypeDef Led) | Toggle a led |
| int32_t BSP_LED_GetState (Led_TypeDef Led) | Return a led state |
| int32_t BSP_PB_Init (Button_TypeDef Button, ButtonMode_TypeDef Button_Mode) | Initialize and configure a button |
| int32_t BSP_PB_DeInit (Button_TypeDef Button) | De-initialize a Button |
| int32_t BSP_PB_GetState (Button_TypeDef Button) | Return a button state |
| void BSP_PB_Callback (Button_TypeDef Button) | Button callback when used in EXTI mode |


© BME-MIT 2026

# Példa az absztrakcióra: LED kezelés

**Header file**

```
typedef enum
{
  LED3 = 0,
  LED4 = 1
}Led_TypeDef;

#define LEDn                                    2

#define LED3_PIN                                GPIO_PIN_13
#define LED3_GPIO_PORT                          GPIOG
#define LED3_GPIO_CLK_ENABLE()                  __HAL_RCC_GPIOG_CLK_ENABLE()
#define LED3_GPIO_CLK_DISABLE()                 __HAL_RCC_GPIOG_CLK_DISABLE()

#define LED4_PIN                                GPIO_PIN_14
#define LED4_GPIO_PORT                          GPIOG
#define LED4_GPIO_CLK_ENABLE()                  __HAL_RCC_GPIOG_CLK_ENABLE()
#define LED4_GPIO_CLK_DISABLE()                 __HAL_RCC_GPIOG_CLK_DISABLE()
```

Absztrakt LED elnevezés

# Példa az absztrakcióra: LED kezelés

**Header file**

```
typedef enum
{
  LED3 = 0,
  LED4 = 1
}Led_TypeDef;

#define LEDn                                2

#define LED3_PIN                            GPIO_PIN_13
#define LED3_GPIO_PORT                      GPIOG
#define LED3_GPIO_CLK_ENABLE()              __HAL_RCC_GPIOG_CLK_ENABLE()
#define LED3_GPIO_CLK_DISABLE()             __HAL_RCC_GPIOG_CLK_DISABLE()

#define LED4_PIN                            GPIO_PIN_14
#define LED4_GPIO_PORT                      GPIOG
#define LED4_GPIO_CLK_ENABLE()              __HAL_RCC_GPIOG_CLK_ENABLE()
#define LED4_GPIO_CLK_DISABLE()             __HAL_RCC_GPIOG_CLK_DISABLE()
```

Absztrakt LED elnevezés

Hardware kapcsolat a LED-el

# Példa az absztrakcióra: LED kezelés

**Forrás file**

```
GPIO_TypeDef* GPIO_PORT[LEDn] = {LED3_GPIO_PORT, LED4_GPIO_PORT};

const uint16_t GPIO_PIN[LEDn] = {LED3_PIN, LED4_PIN};

void BSP_LED_Init(Led_TypeDef Led)
{
  GPIO_InitTypeDef  GPIO_InitStruct;

  /* Enable the GPIO_LED Clock */
  LEDx_GPIO_CLK_ENABLE(Led);

  /* Configure the GPIO_LED pin */
  GPIO_InitStruct.Pin = GPIO_PIN[Led];
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  GPIO_InitStruct.Speed = GPIO_SPEED_FAST;

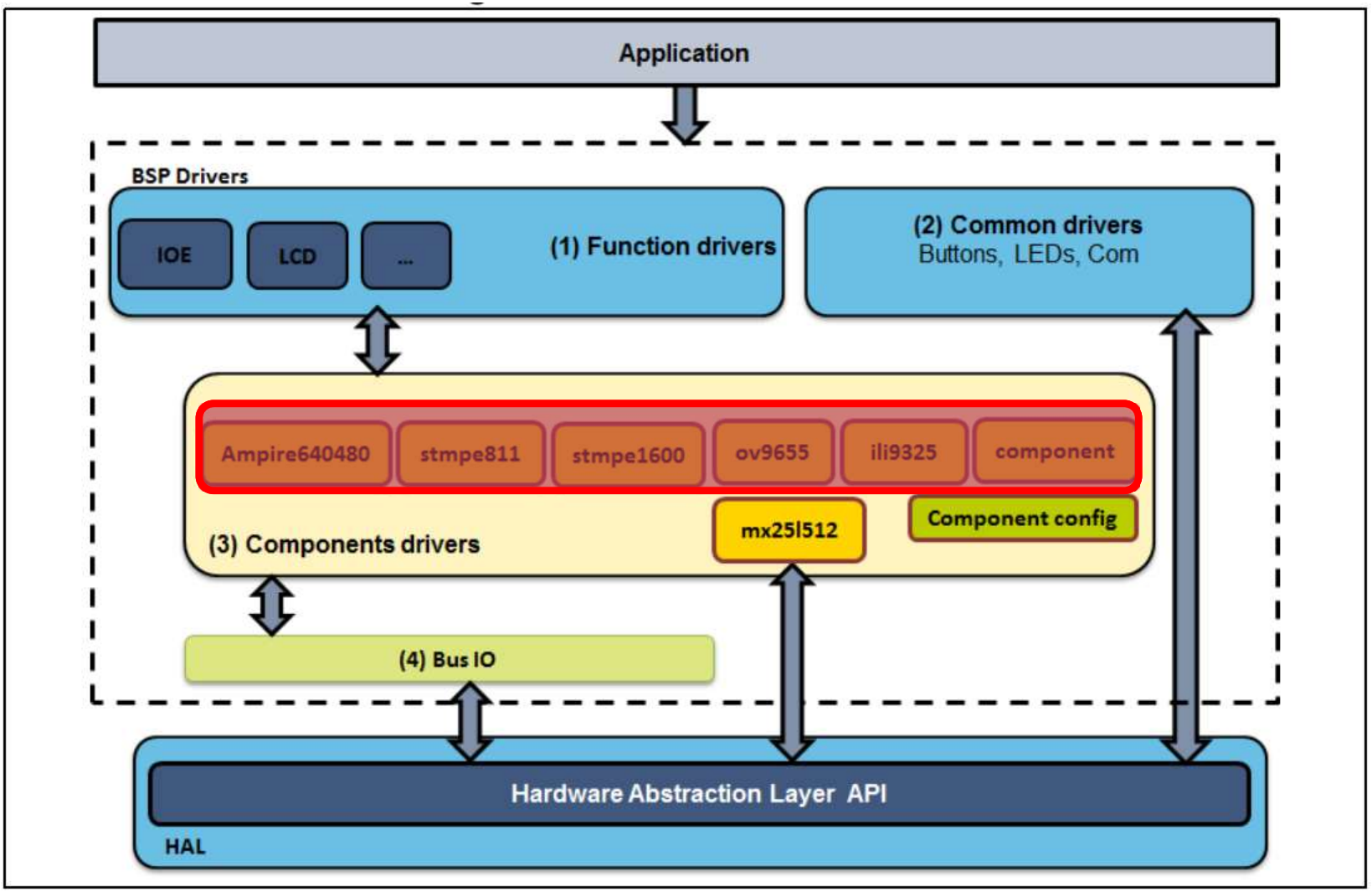
  HAL_GPIO_Init(GPIO_PORT[Led], &GPIO_InitStruct);

  HAL_GPIO_WritePin(GPIO_PORT[Led], GPIO_PIN[Led], GPIO_PIN_RESET);
}
```

Hardware független LE kezelés

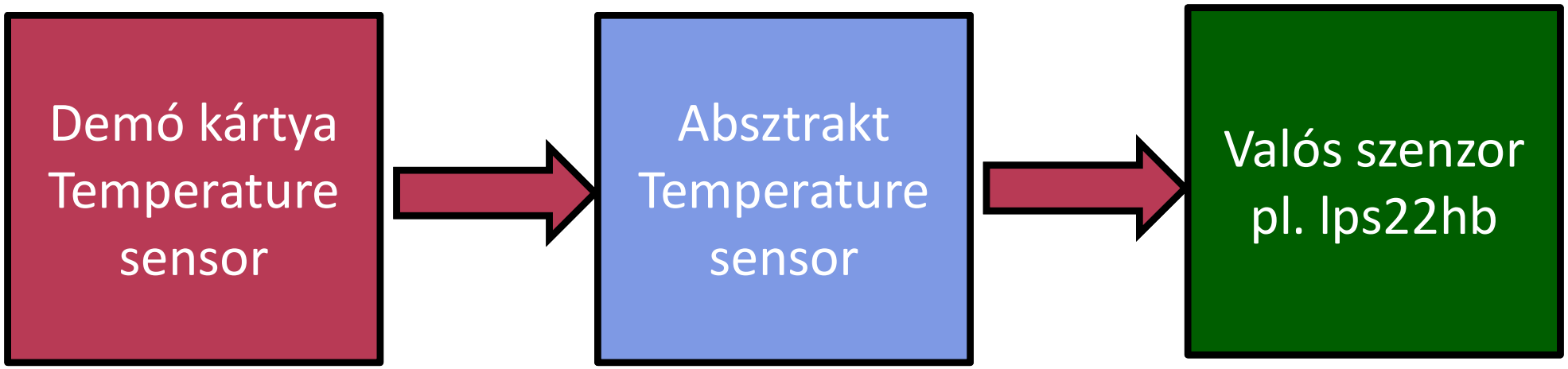
# BSP felépítés részletes példa ST Micro

- Látható, hogy itt is több szintre tagolódik maga a BSP funkcionalítás



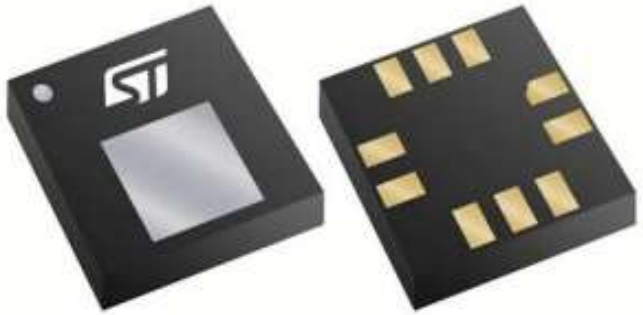
# Komponensek szétbontása több részre

- Egy tényleges fizikai bekötés -> egy absztrakt szenzor típus -> egy valós fizikai szenzor



# Példa: LPS22HB nyomás és hőmérő

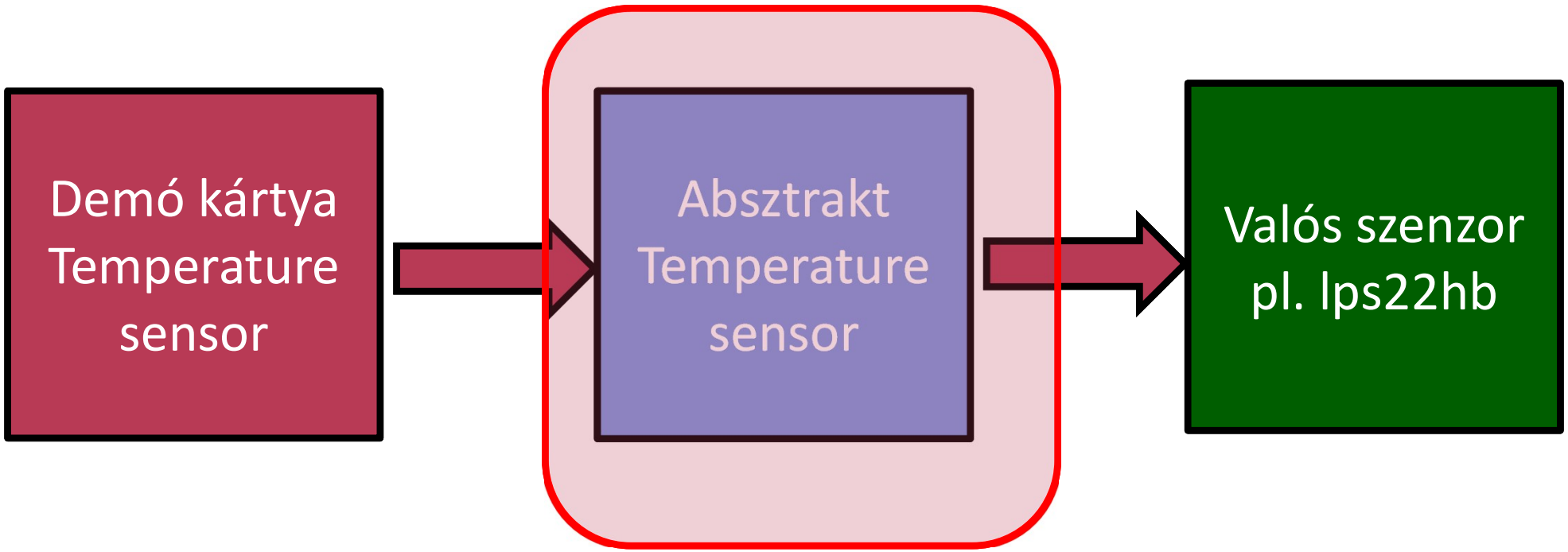
- I2C vagy SPI digitális interfész, általában az I2C preferált
- Külön nyomás és hőmérséklet mérő regiszterek
  - Valóságban a hőmérséklet mindenképp kell a nyomáshoz, ezért elérhetővé teszik azt is
- Mérési tartomány: -40 - +85C
- Digitális skálázás: 100 LSB/°C = 0,01 °C/LSB; abszolút pontosság jóval rosszabb (~±1,5 °C).



**HLGA-10L**
**(2.0 x 2.0 x 0.76 mm)**

# Komponensek szétbontása több részre

- Egy tényleges fizikai bekötés -> egy absztrakt szenzor típus -> egy valós fizikai szenzor



# Absztrakt hőmérő megvalósítás (tsensor.h)

- Célja, hogy bármely hőmérséklet mérő chipet leírjon
  - Gyakorlatilag egy objektum orientált szemléletben egy absztrakt class-ként fogható fel tagváltozókkal és metódusokkal

```
typedef struct
{
  uint8_t AlertMode;             /* Alert Mode Temperature out of range*/
  uint8_t ConversionMode;        /* Continuous/One Shot Mode */
  uint8_t ConversionResolution; /* Temperature Resolution */
  uint8_t ConversionRate;        /* Number of measure per second */
  int8_t TemperatureLimitHigh; /* High Temperature Limit Range */
  int8_t TemperatureLimitLow;  /* Low Temperature Limit Range */
}TSENSOR_InitTypeDef;

typedef struct
{
  void        (*Init)(uint16_t, TSENSOR_InitTypeDef *);
  uint8_t     (*IsReady)(uint16_t, uint32_t);
  uint8_t     (*ReadStatus)(uint16_t);
  float       (*ReadTemp)(uint16_t);
}TSENSOR_DrvTypeDef;
```

# Absztrakt hőmérő megvalósítás (tsensor.h)

- Célja, hogy bármely hőmérséklet mérő chipet leírjon
  - Gyakorlatilag egy objektum orientált szemléletben egy absztrakt class-ként fogható fel tagváltozókkal és metódusokkal

Tagváltozók

```
typedef struct
{
  uint8_t AlertMode;              /* Alert Mode Temperature out of range*/
  uint8_t ConversionMode;         /* Continuous/One Shot Mode */
  uint8_t ConversionResolution; /* Temperature Resolution */
  uint8_t ConversionRate;         /* Number of measure per second */
  int8_t TemperatureLimitHigh;  /* High Temperature Limit Range */
  int8_t TemperatureLimitLow;   /* Low Temperature Limit Range */
}TSENSOR_InitTypeDef;
```

```
typedef struct
{
  void        (*Init)(uint16_t, TSENSOR_InitTypeDef *);
  uint8_t     (*IsReady)(uint16_t, uint32_t);
  uint8_t     (*ReadStatus)(uint16_t);
  float       (*ReadTemp)(uint16_t);
}TSENSOR_DrvTypeDef;
```

# Absztrakt hőmérő megvalósítás (tsensor.h)

- Célja, hogy bármely hőmérséklet mérő chipet leírjon
  - Gyakorlatilag egy objektum orientált szemléletben egy absztrakt class-ként fogható fel tagváltozókkal és metódusokkal

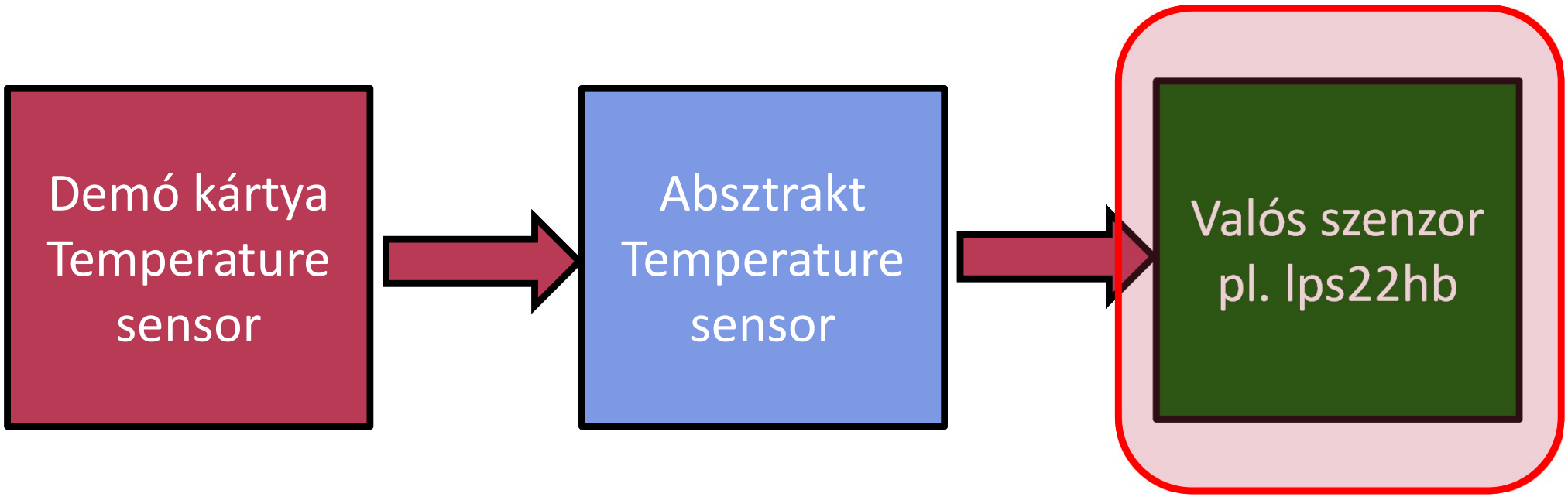
```
typedef struct
{
  uint8_t AlertMode;             /* Alert Mode Temperature out of range*/
  uint8_t ConversionMode;        /* Continuous/One Shot Mode */
  uint8_t ConversionResolution; /* Temperature Resolution */
  uint8_t ConversionRate;        /* Number of measure per second */
  int8_t TemperatureLimitHigh; /* High Temperature Limit Range */
  int8_t TemperatureLimitLow;  /* Low Temperature Limit Range */
}TSENSOR_InitTypeDef;
```

Metódusok

```
typedef struct
{
  void       (*Init)(uint16_t, TSENSOR_InitTypeDef *);
  uint8_t    (*IsReady)(uint16_t, uint32_t);
  uint8_t    (*ReadStatus)(uint16_t);
  float      (*ReadTemp)(uint16_t);
}TSENSOR_DrvTypeDef;
```

# Komponensek szétbontása több részre

- Egy tényleges fizikai bekötés -> egy absztrakt szenzor típus -> egy valós fizikai szenzor



# Valós szenzor: LPS22HB

- Célja, a driver függvények megvalósítása és funkcióval való kitöltése

```
TSENSOR_DrvTypeDef LPS22HB_T_Drv =
{
  LPS22HB_T_Init,
  0,
  0,
  LPS22HB_T_ReadTemp
};
```

Driver implementáció

```
float LPS22HB_T_ReadTemp(uint16_t DeviceAddr)
{
  uint8_t buffer[2];
  uint8_t i;

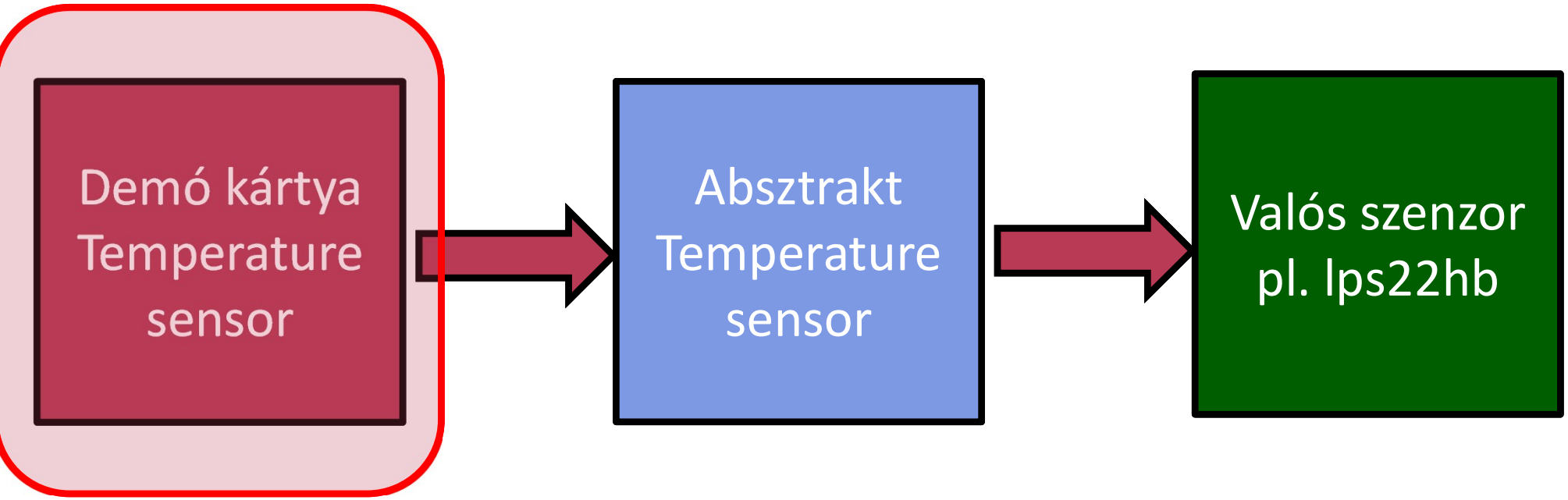
  for(i = 0; i < 2; i++)
  {
    buffer[i] = SENSOR_IO_Read(DeviceAddr, (LPS22HB_TEMP_OUT_L_REG + i));
  }

  /* Build the raw tmp */
  int16_t raw = (int16_t)((buffer[1] << 8) | buffer[0]);
  return  raw / 100.0f;
}
```

Ha egy függvény nem kell, akkor inkább üresen implementáljuk. Ha NULL-t használunk, mint ebben a mintában, akkor a hívó oldalnak hívás előtt ellenőriznie kell, hogy létezik-e a függvény

# Komponensek szétbontása több részre

- Egy tényleges fizikai bekötés -> egy absztrakt szenzor típus -> egy valós fizikai szenzor



# Felhasználás a demókártyán

- Már hardware-hez kötve a részleteket, konfig a header file-ban

```
uint32_t BSP_TSENSOR_Init(void)
{
  uint8_t ret = TSENSOR_ERROR;

#ifdef USE_LPS22HB_TEMP
  tsensor_drv = &LPS22HB_T_Drv;
#else /* USE_HTS221_TEMP */
  tsensor_drv = &HTS221_T_Drv;
#endif

  /* Low level init */
  SENSOR_IO_Init();

  /* TSENSOR Init */
  tsensor_drv->Init(TSENSOR_I2C_ADDRESS, NULL);

  ret = TSENSOR_OK;

  return ret;
}

/**
  * @brief  Read Temperature register of TS751.
  * @retval STTS751 measured temperature value.
  */
float BSP_TSENSOR_ReadTemp(void)
{
  return tsensor_drv->ReadTemp(TSENSOR_I2C_ADDRESS);
}
```

# Tartalom

- Az eddig megismert szoftver megoldások korlátai
- Board Support Package (nem feltétlen linuxi értelemben)
  - Miért szükséges a BSP
  - Jellemző absztrakciós megoldások
    - Define alapú absztrakció
    - Függvény API alapú absztrakció
    - Pointer struktúra alapú absztrakció
  - Ipari példák absztrakciós szintekre, software rétegekre
    - AUTOSAR
    - ST STM BSP
- **Példa absztrakciós szintek meghúzására**

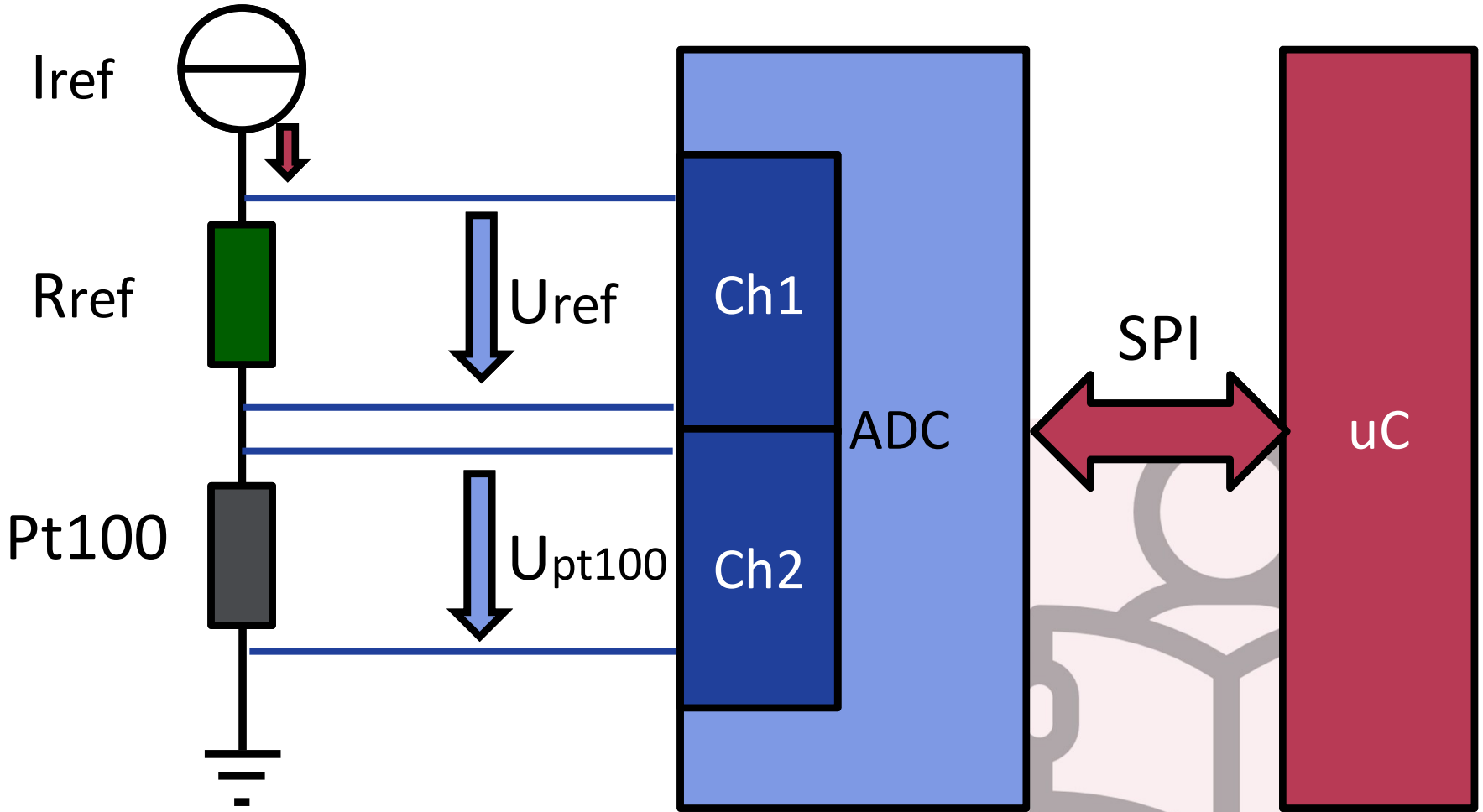

© BME-MIT 2026

# Hőmérsékletet szeretnénk mérni Pt100 ellenállással

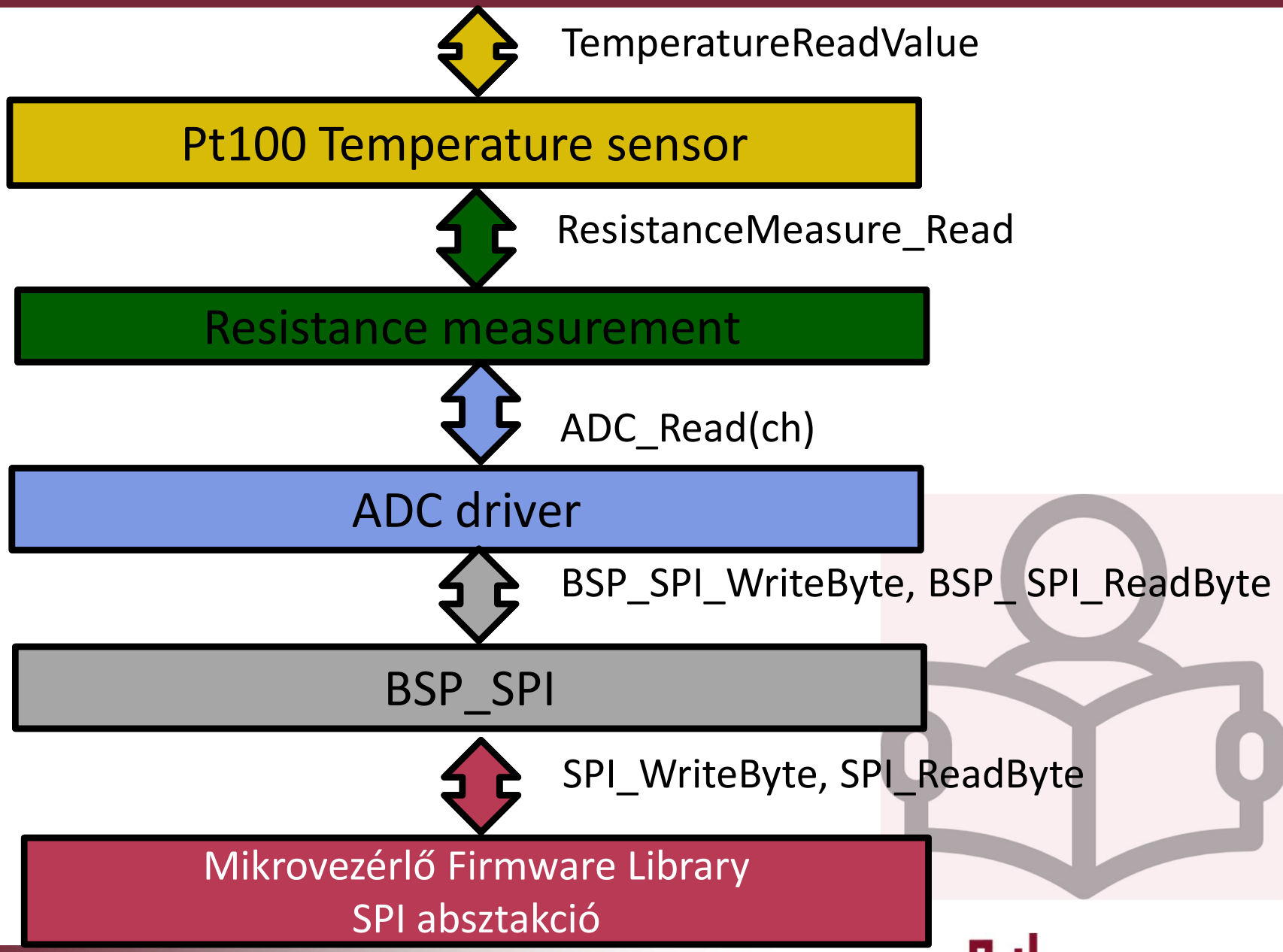
- Nagy hőmérséklet tartomány -200 tól 800+C-ig
- Elég pontos, akár 0.1C
- Viszonylag lineáris, de annyira nem: Callendar–Van Dusen equation
- De elég macerás mérni: 0,385 Ohm/C a változása ami elég kicsi, nagyon gerjeszteni nem szabad, mert akkor attól is melegszik



# Hőmérsékletet szeretnénk mérni Pt100 ellenállással, javasolt elrendezés



# Milyen absztrakciós szinteket raknánk le




© BME-MIT 2026

# Összefoglalás: szoftver rétegek és BSP

- A BSP a board konkrét hardverét (lábkiosztás, külső komponensek) írja le és takarja el; a firmware library-re épül, a felsőbb rétegek már csak a BSP-t látják.
- Függőség csak lefelé: a felsőbb réteg hívja az alsót, visszajelzés felfelé visszatérési értékkel vagy callbackkel.
- Az ipari megoldások (AUTOSAR, ST BSP) ugyanezeket a technikákat kombinálják, több absztrakciós szinten.

| | Define alapú | Függvény API alapú | Pointer struktúra alapú |
|---|---|---|---|
| **Mit takar el** | Lábkiosztás, varázskonstansok | A firmware library hívásokat | A konkrét implementációt |
| **Hordozhatóság** | Board-változatok között | MCU-váltáskor is (API mögött) | Legjobb, akár futásidejű választás |
| **Bonyolultság** | Alacsony | Közepes | Magas |
| **Futásidejű költség** | Nincs | Minimális | Indirekt hívás |
| **Mikor érdemes** | Lábak, konstansok | A legtöbb board funkció | Több implementáció azonos interfésszel |


© BME-MIT 2026